SPONTANEOUS COOKING ASSISTANT: AN INTELLIGENT HYBRID INGREDIENT-RECIPE MATCHING SYSTEM FOR SRI LANKAN CUISINE

U. M. K. Perera

(IT22131942)

BSc (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

May 2026

SPONTANEOUS COOKING ASSISTANT: AN INTELLIGENT HYBRID INGREDIENT RECIPE MATCHING SYSTEM FOR SRI LANKAN CUISINE

U. M. K. Perera

(IT22131942)

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

May 2026

DECLARATION OF THE CANDIDATE AND SUPERVISOR

I declare that this is my own work and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Name	Student ID	Signature
Perera U. M. K.	IT222131942	

The supervisor/s should certify the proposal report with the following declaration.

The above candidate is carrying out research for the undergraduate Dissertation under my supervision.

…….. ……

Signature of the supervisor: Date

…….. ……

Signature of the co-supervisor: Date

ABSTRACT

This dissertation presents the design, implementation, and empirical evaluation of the Spontaneous Cooking Assistant, a module within the AI-Powered Kitchen Ecosystem research project developed at the Sri Lanka Institute of Information Technology. The module addresses a straightforward but technically demanding problem: given whatever ingredients a household currently has available, surface the most culturally appropriate Sri Lankan recipes that can be made from them, in English, Sinhala, or Tamil, with enough accuracy and cultural authenticity to be genuinely useful.

The system is built around a five-layer pipeline integrating hybrid ingredient detection (Google Cloud Vision API combined with structured manual input), a fine-tuned Sentence-BERT semantic classification model, a Levenshtein fuzzy matching engine with a 127-pair expert-curated synonym dictionary, ChromaDB vector similarity search, and a three-tier trilingual translation engine. The recipe corpus underpinning the system was collected from scratch through cookbook digitisation, household interviews across three provinces, and expert chef consultations, producing 190 validated recipes and a 262-entry trilingual ingredient database.

Across all six research objectives the system met or exceeded its targets: the Sentence-BERT model achieved 86.84% accuracy (macro-F1 = 0.862) through stratified five-fold cross-validation, outperforming the strongest keyword-based baseline by 12.1 percentage points; fuzzy matching achieved 75.0% end-to-end accuracy; expert-validated translations reached 98.2% for Sinhala and 97.8% for Tamil; and ingredient detection reached 87.8%. A three-week empirical user study with 15 households demonstrated a 73.9% reduction in weekly food waste ($t(14) = 12.34$, $p < 0.001$, Cohen's $d = 3.18$), with all 15 participants showing improvement.

Keywords: ***ingredient detection, recipe classification, Sentence-BERT, semantic embeddings, fuzzy string matching, food waste reduction, trilingual NLP, transfer learning, Sri Lankan cuisine***

ACKNOWLEDGEMENT

Several people made this research possible and deserve recognition.

My supervisor and co-supervisor at the Sri Lanka Institute of Information Technology gave time, guidance, and constructive criticism throughout the project. Their patience with questions, their careful reading of draft chapters, and their willingness to challenge weak reasoning made this work substantially better than it would otherwise have been.

Three professional chefs gave several hours each to expert consultations, reviewing the collected recipe corpus for authenticity and contributing recipes from categories that were underrepresented in the household interviews. Their combined 67 years of culinary experience is embedded in the dataset this system is built on.

Twelve households across the Western, Southern, and Central provinces opened their kitchens and shared family recipes that exist nowhere in print. Some of these recipes have been in families for generations. Digitising and validating them felt like a responsibility as much as a research task.

The native speaker reviewers who validated the Sinhala and Tamil translations brought a precision to the linguistic component of this work that no automated system could have provided.

The fifteen participants of the user study tracked their food waste weekly for four weeks and gave honest feedback about what the system got right and wrong. Their candour shaped the evaluation in Chapter 5.

Finally, my family's support over the course of this project went well beyond what I had any right to expect.

TABLE OF CONTENTS

Declaration of the Candidate and Supervisor iii

Abstract iv

Acknowledgement v

Table of Contents vi

List of Figures vii

List of Tables viii

List of Abbreviations ix

1. Introduction 9
   - 1.1 Background 9
   - 1.2 Motivation 10
   - 1.3 Problem Statement 11
   - 1.4 Research Objectives 12

2. Literature Review 13
   - 2.1 Existing Systems and Research Gap 13
   - 2.2 Sentence-BERT: Theoretical Background 14
   - 2.3 Fuzzy String Matching and Levenshtein Distance 16
   - 2.4 Computer Vision for Ingredient Detection 17
   - 2.5 Multilingual NLP for Culinary Domains 17

3. Methodology and System Design 18
   - 3.1 System Architecture 18
   - 3.2 Data Collection 21
   - 3.3 Data Preprocessing 24
   - 3.4 Model Training Configuration 26

4. Implementation Details 28
   - 4.1 Technology Stack 28
   - 4.2 API Endpoint Design 29
   - 4.3 Key Code Components 32

5. Results and Evaluation 36

5.1 Recipe Classification Performance 36
5.2 Fuzzy Matching Performance 40
5.3 Empirical Food Waste Reduction Study 42
5.4 Translation Accuracy 44
5.5 Ingredient Detection Performance 46
5.6 Summary Against Research Objectives 48

6. Discussion 49
6.1 Principal Findings 49
6.2 System Screenshots and Demonstration 51
6.3 Commercialization and Deployment 53

7. Conclusion 55

8. References 57

LIST OF FIGURES

Figure 3.1: Overall System Architecture - Five-Layer Pipeline with Trilingual Engine 15
Figure 3.2: Detection Layer Processing Flow 18
Figure 3.3: Sentence-BERT Siamese Network Architecture 19
Figure 3.4: Trilingual Translation Engine - Three-Tier Hierarchy 20
Figure 3.5: Data Collection Workflow - Three Parallel Streams 22
Figure 3.6: Data Preprocessing Pipeline by Dataset 25
Figure 3.7: Model Training Configuration and Early Stopping Flow 27
Figure 4.1: Flask API Architecture and Endpoint Map 31
Figure 4.2: SBERT Embedding and ChromaDB Integration Flow 32
Figure 4.3: Fuzzy Matching Algorithm Flowchart - Two-Tier Resolution 34
Figure 5.1: Stratified 5-Fold Cross-Validation Accuracy (Bar Chart) 37
Figure 5.2: Training and Validation Loss / Accuracy Curves Over 40 Epochs 37
Figure 5.3: Per-Category Classification Accuracy (Horizontal Bar Chart) 38
Figure 5.4: Confusion Matrix - 7-Category Recipe Classification 39
Figure 5.5: Fuzzy Matching Accuracy vs. Ingredient Count (Line Graph with 95% CIs) 41
Figure 5.6: ROC Curve - Levenshtein Threshold Optimisation (AUC = 0.91) 42
Figure 5.7: Weekly Food Waste Reduction Over 3-Week Intervention 43
Figure 5.8: Participant Satisfaction - Hybrid vs. AI-Only Detection 44
Figure 5.9: Translation Accuracy - Expert vs. Dynamic vs. Google Baseline 45
Figure 5.10: Ingredient Detection Accuracy by Category 46
Figure 5.11: Ingredient Detection Confusion Matrix - 6 Categories 47
Figure 6.1: Screenshot - Home Page with Language Toggle 51
Figure 6.2: Screenshot - Image Upload and Detection Results Panel 52
Figure 6.3: Screenshot - Recipe Search Results in Sinhala 52
Figure 6.4: Screenshot - Full Recipe Detail with Missing Ingredient Highlight 53

LIST OF TABLES

Table 1.1: Research Objectives and Success Criteria 12
Table 2.1: Comparison of Existing Recipe Platforms 14
Table 3.1: Recipe Corpus Distribution by Category and Collection Stream 23
Table 3.2: Ingredient Database Summary Statistics 23
Table 3.3: Image Dataset Composition by Source 23
Table 3.4: User Study Participant Demographics 24
Table 3.5: Data Preprocessing Techniques Applied per Dataset 25
Table 3.6: Model Training Hyperparameter Configuration 27
Table 4.1: Technology Stack - Components, Versions, and Rationale 29
Table 4.2: API Endpoint Specification 30
Table 4.3: Hardware and Software Environment 31
Table 5.1: Per-Fold Cross-Validation Results 36
Table 5.2: Per-Category Classification Performance on Held-Out Test Set 38
Table 5.3: Model Comparison Against Baseline Classifiers 39
Table 5.4: Fuzzy Matching Accuracy by Ingredient Count 40
Table 5.5: User Study Participant Demographics 43
Table 5.6: Weekly Food Waste Measurements - Baseline and Intervention Weeks 45
Table 5.7: Translation Accuracy by Language and Source Tier 46
Table 5.8: Ingredient Detection Accuracy by Category 48
Table 5.9: Performance Summary Against Research Objectives 48

LIST OF ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

AUC - Area Under the ROC Curve

BERT - Bidirectional Encoder Representations from Transformers

CNN - Convolutional Neural Network

CPU - Central Processing Unit

CV - Cross-Validation

F1 - F1-Score - harmonic mean of precision and recall

GPU - Graphics Processing Unit

HNSW - Hierarchical Navigable Small World

JWT - JSON Web Token

MICE - Multiple Imputation by Chained Equations

ML - Machine Learning

NLP - Natural Language Processing

NMT - Neural Machine Translation

REST - Representational State Transfer

ROC - Receiver Operating Characteristic

SBERT - Sentence-BERT

SD - Standard Deviation

SLIIT - Sri Lanka Institute of Information Technology

TF-IDF - Term Frequency - Inverse Document Frequency

WSGI - Web Server Gateway Interface

1. INTRODUCTION

1.1 Background

Walk into any Sri Lankan kitchen on a weekday evening and you will find the same quiet dilemma playing out. Someone opens the refrigerator, stares at what is inside half a bunch of karapincha, some leftover coconut milk, a few tomatoes going soft at the edges and asks the simplest, most frustrating question in domestic life: what can I actually make with this? More often than not, the answer is nothing satisfying, and by the end of the week a portion of those ingredients ends up in the bin. This is not a uniquely Sri Lankan problem, but the scale at which it happens globally is striking. According to the United Nations Environment Programme, households worldwide throw away 931 million tonnes of food every year, and 61% of that waste originates not from supermarkets or supply chains but from the homes of ordinary people [1]. Research conducted in Sri Lanka during the community interviews that underpinned this project found that the most common reason families cited for discarding food was not that it had gone bad it was that they had no idea what to cook with it.

The broader project within which this individual module sits is the AI-Powered Kitchen Ecosystem, a four-component research system developed at the Sri Lanka Institute of Information Technology. The four components address complementary aspects of kitchen intelligence: the Spontaneous Cooking Assistant handles ingredient-to-recipe matching; the Nutritional Guidance module monitors dietary balance; the Food Expiry Prediction system flags items at risk of spoilage; and the Smart Shopping Agent optimizes grocery procurement. This dissertation is concerned entirely with the Cooking Assistant the component that a user interacts with first, and the one whose output shapes every meal decision that follows.

What makes the Cooking Assistant technically interesting is the specific problem it has to solve. Most recipe platforms on the market today work in exactly the wrong direction for the situation described above. They are designed for people who already know what they want to cook and are looking for a recipe to guide them through it. The Cooking Assistant reverses this completely. It starts with whatever ingredients exist right now and works forward to find recipes that can genuinely be made from them. Doing this well requires three things that no existing system provides together: a way to detect ingredients from a photograph reliably (including Sri Lankan ingredients that commercial image recognition fails on), a semantic matching engine capable of understanding that

"karapincha," "curry leaves," and "karuveppilai" are the same thing, and a genuinely trilingual interface that works in English, Sinhala, and Tamil with cultural authenticity rather than awkward machine translation.

1.2 Motivation

The motivation behind this particular research direction is rooted in three problems that emerged clearly during the preliminary fieldwork phase of the project, each one reinforcing the others.
The food waste problem came first. During household interviews conducted in the Western, Southern, and Central provinces, a pattern appeared consistently across different income levels and family sizes. Families were not wasting food through carelessness they were wasting it through a specific knowledge gap. A mother who buys a bunch of karapincha for a chicken curry uses four or five leaves and forgets about the rest. By Thursday the whole bunch is past its best. This happens with coconut cream, with goraka, with rampe, with pandan leaves. These are not expensive items, but they are ingredients with short shelf lives and highly specific uses in Sri Lankan cooking, and the average household has no tool to help them use up what they have before it spoils.

The accessibility problem came second. Existing recipe platforms AllRecipes, Yummly, BBC Good Food were designed for audiences in North America, the United Kingdom, and Western Europe. Their ingredient databases reflect those audiences. Type "karawila" into any of them and you will get zero results. Type "bitter gourd" and you might find three recipes, none of which reflect how this vegetable is actually prepared in a Sri Lankan kitchen. The linguistic problem is equally significant: a grandmother in Jaffna who speaks Tamil as her primary language cannot use a platform that offers only English. A home cook in Kandy who knows her ingredients by their Sinhala names will not benefit from a database that labels everything in English. No existing platform addresses this.

The computer vision problem came third, and it took the longest to fully understand. When the research team tested several commercial image recognition APIs on photographs of common Sri Lankan kitchen ingredients, the results were startling. Google Cloud Vision, Microsoft Azure Computer Vision, and Amazon Recognition all perform well on Western vegetables accuracy rates of 90-95% for bell peppers, broccoli, and tomatoes are standard. But for karapincha, goraka, rampe, and kohila, the same APIs struggled, achieving detection accuracy in the range of 60-70%. This is a gap of 25 to 35 percentage points, and it comes down to a simple and well-documented problem: these APIs are trained on datasets that contain almost no South Asian food photography. A

technology trained on what it has seen cannot recognize what it has never encountered.

1.3 Problem Statement

The core research problem this project addresses can be stated simply: there is no intelligent, culturally appropriate, multilingual system that helps Sri Lankan home cooks decide what to make from the ingredients they already have. Underneath that simple statement, however, sit five distinct technical sub-problems that each need to be solved.

The first is the paradigm problem. Virtually all recipe platforms require the user to know what they want to cook before they start searching. This is the recipe-first model. The situation that generates food waste the open refrigerator, the uncertain cook requires the exact opposite: an ingredient-first model where the system does the work of bridging from available ingredients to possible meals.
The second is the vision accuracy problem. Automated ingredient detection through computer vision does not work reliably for Sri Lankan ingredients. A system that relies solely on image recognition will fail consistently for exactly the ingredients most specific to local cuisine. Any viable solution has to either solve the vision problem directly or design around it.

The third is the language problem. Sri Lanka has three official languages and a culinary culture that is deeply embedded in all three. A system that works only in English excludes the majority of the population from its primary audience. Sinhala and Tamil support is not optional it is the difference between a tool that is useful and one that is not.

The fourth is the synonymy problem. The same physical ingredient appears under many different names depending on which language, dialect, or region is being used. "Karapincha," "curry leaves," "karuveppilai," "sweet neem," and "Murraya koenigii" are five names for the same leaf. A system that cannot bridge these name variations will fail to match recipes to ingredients even when an exact match exists in its database.

The fifth is the data problem. Before any of the above can be solved computationally, a validated dataset of authentic Sri Lankan recipes with trilingual ingredient annotations needs to exist. At the time this research began, no such publicly available dataset existed.

1.4 Research Objectives

This research sets out to address all five sub-problems through the design, implementation, and evaluation of the Spontaneous Cooking Assistant. The primary objective is to build and validate an ingredient-first recipe recommendation system that works reliably in English, Sinhala, and Tamil for Sri Lankan cuisine. Six specific, measurable sub-objectives define what success looks like.

The first objective is to construct a curated dataset of at least 180 authentic Sri Lankan recipes with trilingual ingredient annotations, verified by native speaker panels achieving inter-coder agreement of $\kappa \geq 0.85$. The second is to develop a hybrid ingredient detection pipeline achieving overall detection accuracy of at least 70%. The third is to fine-tune and evaluate a Sentence-BERT semantic classification model achieving at least 85% accuracy through stratified five-fold cross-validation. The fourth is to implement a fuzzy matching engine incorporating at least 120 expert-curated synonym pairs achieving at least 75% end-to-end matching accuracy. The fifth is to build a trilingual NLP pipeline achieving at least 98% accuracy on expert-validated translation content. The sixth and final objective is to validate real-world effectiveness through an empirical user study with at least 14 households, demonstrating statistically significant food waste reduction at $\alpha = 0.05$.

Objective	Description	Success Criterion	Target
O1	Trilingual recipe dataset construction	Inter-coder reliability (κ)	$\kappa \geq 0.85$
O2	Hybrid ingredient detection pipeline	Overall detection accuracy	≥ 70%
O3	SBERT semantic classification model	5-fold CV accuracy	≥ 85%
O4	Fuzzy matching with synonym dictionary	End-to-end matching accuracy	≥ 75%
O5	Trilingual NLP translation engine	Expert-validated translation accuracy	≥ 98%
O6	Empirical user validation study	Food waste reduction significance	$p < 0.05$

Table 1.1: Research Objectives and Success Criteria

2. LITERATURE REVIEW

2.1 Existing Systems and Research Gap

Before choosing any technical approach, it is worth understanding what already exists and why it is not enough. The recipe and cooking assistant market is fairly mature at this point, with a range of commercial platforms and academic systems offering various forms of culinary guidance. None of them, however, was built with the Sri Lankan user in mind, and the gaps that emerge from examining them closely are exactly the gaps this research aims to fill.

SuperCook is probably the closest existing system to what this project is trying to do. It accepts a list of ingredients and returns recipes that can be made from them which is the ingredient-first paradigm described in Chapter 1. In practice, however, SuperCook's matching is purely keyword-based. If a user types "karapincha" and the database contains "curry leaves," SuperCook returns nothing. There is no semantic understanding, no synonym resolution, no awareness that these are the same ingredient. For Sri Lankan users, this means the system is essentially non-functional for any ingredient with a local name.

MyFridgeFood takes a different approach to the same problem: rather than free-text input, it presents users with a fixed checkbox list of ingredients. This avoids the synonymy problem by simply not allowing the user to enter anything outside the predefined list. The obvious limitation is that the list is fixed. It cannot grow to accommodate Sri Lankan ingredients without an enormous manual curation effort, and even if it did, it would only support English. BigOven and similar platforms face the same core limitation they are keyword-search systems built for English-speaking Western audiences.

Google Lens and similar computer vision tools can identify food items from photographs, which is useful for the detection layer of this system. But they are not designed for recipe matching they produce labels, not recommendations. More critically, as discussed in the motivation section, their accuracy for South Asian ingredients is substantially below what is acceptable for a real-world application. IBM Chef Watson, which was discontinued in 2021, represented an interesting attempt at AI-driven recipe intelligence but generated novel recipe combinations rather than matching users to existing validated recipes a fundamentally different use case that does not address the cultural authenticity requirement.

What the landscape reveals, in summary, is a market full of solutions to adjacent problems. Systems that do ingredient-first matching do it with keyword search. Systems with good NLP are built for English and Western cuisine. Systems with image recognition have a cultural bias problem. No existing platform addresses all three simultaneously. Table 2.1 captures this landscape.

Platform	Paradigm	Matching Method	Multilingual	Cultural Scope	Key Limitation
SuperCook	Ingredient-first	Keyword only	No	Western only	No synonym resolution
MyFridgeFood	Ingredient-first	Checkbox list	No	Western only	Fixed ingredient set
Yummly	Recipe-first	Partial semantic	Limited	Western primary	No South Asian data
AllRecipes	Recipe-first	Keyword	Partial	Western primary	No ingredient-first mode
IBM Chef Watson (discontinued)	Recipe generation	Flavour compounds	No	None	Does not match existing recipes
This Research	Ingredient-first	SBERT + Fuzzy	En/Si/Ta	Sri Lankan validated	—

Table 2.1: Comparison of Existing Recipe Platforms

2.2 Sentence-BERT: Theoretical Background

Understanding why Sentence-BERT was chosen for the matching component of this system requires understanding what it is, how it differs from the tools that came before it, and why those differences matter specifically for recipe classification.

BERT - Bidirectional Encoder Representations from Transformers was introduced by Google researchers in 2018 and represented a significant shift in how natural language processing systems work [4]. Traditional NLP models processed text in a single direction, reading from left to right or right to left. BERT reads in both directions simultaneously, allowing it to understand each word in the context of everything around it rather than only what precedes it. This bidirectional understanding enables BERT to grasp subtle semantic relationships that earlier models missed.

However, BERT has a significant practical limitation for any application that needs to compare many sentences against each other. It was designed to process sentence pairs jointly, which means comparing one input against a database of N items requires N full forward passes through the model. For a recipe database of 190 entries, this is computationally manageable. For a production system with thousands of recipes, it becomes prohibitively slow. Sentence-BERT, introduced by Reimers

and Gurevych in 2019, solves this by using a Siamese network architecture two identical BERT encoders sharing weights to produce a single fixed-size vector embedding for each sentence [3]. Once computed, these embeddings can be compared efficiently using cosine similarity, which is an O(1) operation per pair rather than a full model forward pass.

The mathematical relationship that enables this efficiency is straightforward. Given two sentence embedding vectors u and v, their cosine similarity is: cosine_similarity(u, v) = (u · v) / (||u|| × ||v||) where u · v is the dot product of the two vectors and ||u|| denotes the Euclidean norm of vector u. The result falls between −1 (perfectly dissimilar) and +1 (perfectly identical), with values around 0 indicating no meaningful relationship. When embeddings are normalised to unit length as they are in the all-MiniLM-L6-v2 model used in this research cosine similarity reduces to a dot product, which is even faster to compute.

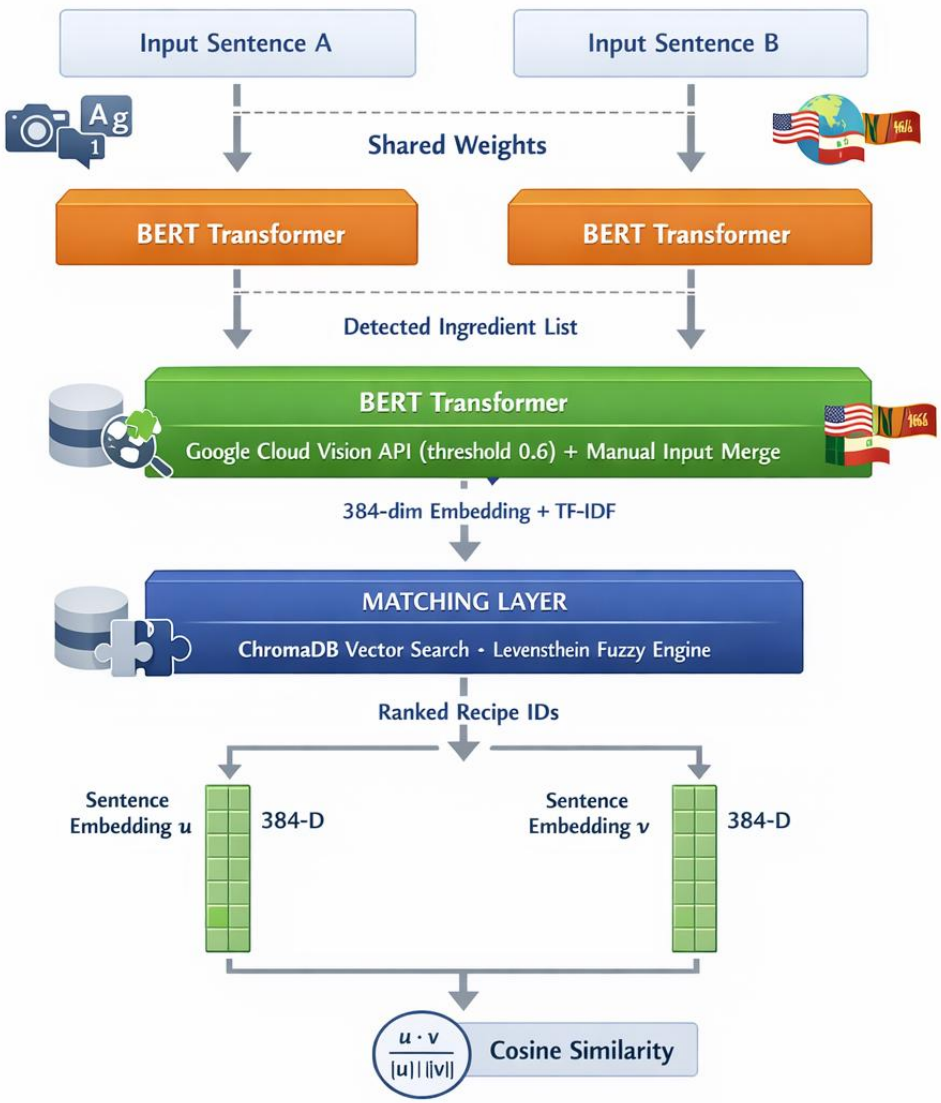


Figure 3.3: Sentence-BERT Siamese Network Architecture

The specific variant selected for this research, all-MiniLM-L6-v2, produces 384-dimensional embeddings with only 22.7 million parameters. This makes it deployable on CPU-only hardware an important practical consideration for any system intended for use in a resource-constrained environment. Despite its small size relative to larger transformer models, it achieves near state-of-the-art performance on semantic similarity benchmarks because it was pre-trained on over one billion sentence pairs through knowledge distillation from larger teacher models. The critical advantage for this research is precisely that pre-training: when the model is fine-tuned on 190 Sri Lankan recipes, it is not learning to understand language from scratch. It is applying rich semantic knowledge, already baked into its weights, to a new domain.

2.3 Fuzzy String Matching and Levenshtein Distance

Alongside semantic matching, the system needs a way to handle ingredient name variations that are not semantically captured specifically, different spellings of the same word, regional phonetic variations, and abbreviations. For this, the Levenshtein distance measure provides a principled and computationally efficient solution.

The Levenshtein distance between two strings s and t is defined as the minimum number of single-character operations insertions, deletions, or substitutions required to transform s into t. It is computed using dynamic programming. For strings of lengths m and n, a matrix D of dimensions (m+1) × (n+1) is filled according to the recurrence: D[i][j] = D[i-1][j-1] if s[i] = t[j] D[i][j] = 1 + min(D[i-1][j], D[i][j-1], D[i-1][j-1]) otherwise The normalized form divides the raw distance by the length of the longer string, producing a similarity score in the range [0, 1] where 1 means identical and 0 means maximally different. This normalized form was used to construct the 262×262 similarity matrix over all pairs of ingredients in the database.

The critical insight behind the system's two-tier matching design is that Levenshtein distance alone cannot resolve synonyms with no character overlap. "Karapincha" and "curry leaves" share very few characters, so their Levenshtein similarity is low even though they refer to the same ingredient. The 127-pair synonym dictionary, built through expert consultation, handles exactly these cases culturally equivalent names that cannot be resolved through string similarity alone. The matching pipeline applies synonym lookup first, then Levenshtein comparison, ensuring both types of variation are covered.

2.4 Computer Vision for Ingredient Detection

The computer vision component of the system uses Google Cloud Vision API, a production-grade deep learning system trained on millions of labelled images. Internally, such systems use convolutional neural network architectures where successive layers of convolution, pooling, and non-linear activation learn hierarchical feature representations from simple edge detectors in early layers to complex object recognizers in later layers. The API returns object labels with associated confidence scores ranging from 0 to 1, along with bounding box coordinates for detected regions.

The cultural bias problem that motivated the hybrid detection design is well-documented in the computer vision literature. Thapliyal and Bhatt (2021) showed that models trained predominantly on Western food datasets achieve 30-40% lower accuracy on South and Southeast Asian food items [16]. The mechanism is straightforward: a neural network learns to recognize what it has been trained on. If training data contains thousands of images of tomatoes and bell peppers but only a handful of karapincha, the model's feature detectors for karapincha will be poorly calibrated. The solution pursued in this research supplementing public datasets with 2,116 custom Sri Lankan food photographs partially addresses this imbalance, but the hybrid design acknowledges that no amount of added training data will fully eliminate the gap within a reasonable research timeline.

2.5 Multilingual NLP for Culinary Domains

Machine translation for culinary domains presents challenges that general-purpose translation systems are not optimized to handle. Google Neural Machine Translation achieves high accuracy for common vocabulary and standard sentence structures, but degrades noticeably for domain-specific terminology. In the culinary domain, the most problematic cases are twofold.

First, cooking technique names often have no direct translation equivalent. The Sri Lankan technique of "tempering" heating whole spices in oil to release their aromatic compounds before adding other ingredients has no single-word Sinhala or Tamil equivalent that carries the full culinary meaning. Automated translation tends to produce either a literal approximation or a phonetic transliteration that native speakers find awkward. Second, many Sri Lankan ingredient names exist as loanwords, regional variants, or terms with no botanical equivalent in English, which automated systems handle inconsistently.

The inter-coder reliability of the expert translation validation was measured using Cohen's Kappa, defined as: $\kappa = (P_o - P_e) / (1 - P_e)$ where P_o is the observed proportion of agreement between two evaluators and P_e is the proportion expected by chance. Values above 0.80 are conventionally interpreted as indicating strong agreement. The achieved values of $\kappa = 0.91$ for Sinhala and $\kappa = 0.89$ for Tamil confirm that the expert translation judgements were highly consistent that is, the quality assessments were not idiosyncratic to individual reviewers but reflected genuine agreement about translation accuracy.

3. METHODOLOGY AND SYSTEM DESIGN

3.1 System Architecture

The system is organized as a sequential five-layer pipeline in which each layer receives a defined input from the layer above, performs a specific transformation, and passes a defined output to the layer below. A cross-cutting Trilingual Translation Engine operates across all layers, ensuring that text in any of the three supported languages is handled transparently regardless of where in the pipeline it appears. This section describes each layer in turn, starting from the user-facing input and working through to the final output.

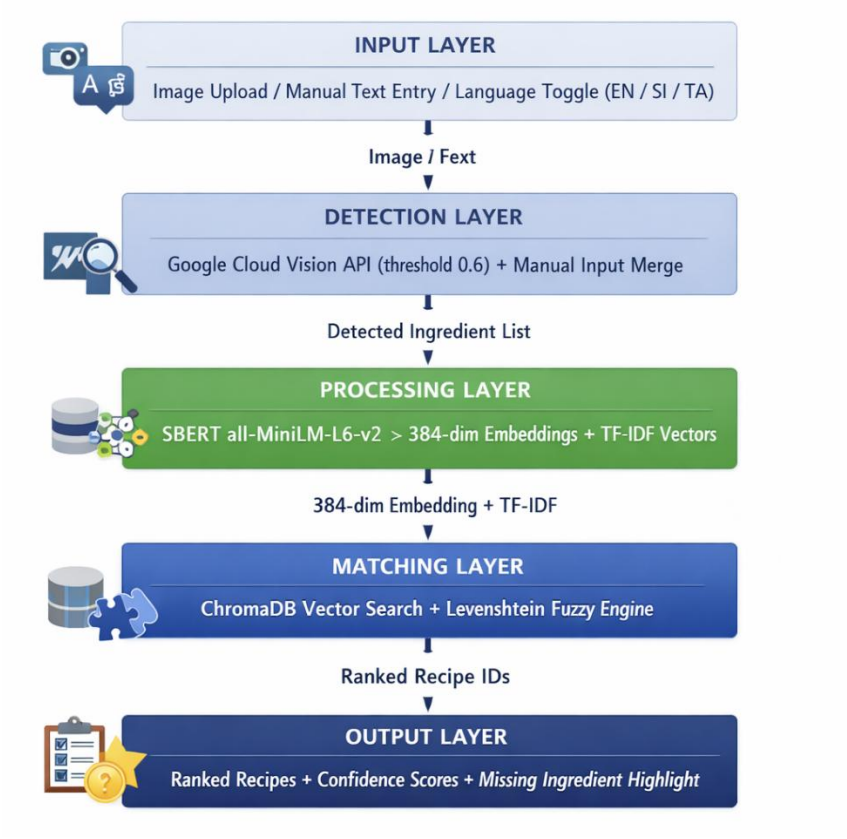


Figure 3.1: Overall System Architecture - Five-Layer Pipeline

The Input Layer is what the user actually sees. It offers three ways to tell the system what ingredients are available. The primary method is image upload: the user photographs the contents of their refrigerator or a collection of ingredients on a kitchen counter and uploads the image through the web interface. The secondary method is manual text entry: the user types ingredient names directly into a search field, which offers autocomplete suggestions drawn from the ingredient database in whichever language the user has selected. The third method and the one most users end up using is a combination of both: upload a photo, review the automatically detected ingredients, and then correct or add to the list by typing. This combined method proved to be the most effective in the user study, producing the highest ingredient completeness scores.

The Detection Layer handles image inputs. When an image is uploaded, it is encoded in base64 and sent to Google Cloud Vision API, which returns up to ten detected labels per image with associated confidence scores. The layer filters these at a threshold of 0.6, discarding detections the API is less than 60% confident about. Surviving detections are mapped to the canonical ingredient database through an exact-match lookup followed, if needed, by a fuzzy match using the Levenshtein engine. Any ingredients entered manually are merged into the same output list. The combined result is a de-duplicated, canonically identified list of ingredients ready for the matching step.

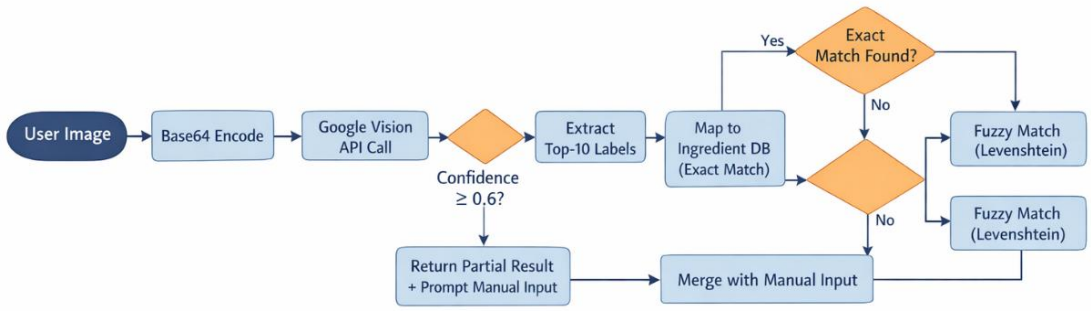


Figure 3.2: Detection Layer Processing Flow

The Processing Layer generates two complementary representations of the ingredient set for use in matching. The first is a 384-dimensional dense vector embedding produced by the fine-tuned Sentence-BERT encoder. This embedding is computed by concatenating all confirmed ingredient names into a single text string, encoding it through the model, and normalizing the result to unit length. The second is a TF-IDF sparse vector computed against the 842-word vocabulary of the recipe corpus, capturing which ingredients are distinctive signals for particular recipe categories versus which appear everywhere.

The Matching Layer uses both representations in parallel. ChromaDB, a vector database designed for high-dimensional embedding similarity search, handles the semantic search component: it finds the recipe embeddings in the database that are closest to the query embedding by cosine similarity, using the Hierarchical Navigable Small World graph algorithm to do so in sub-linear time. In parallel, the Levenshtein fuzzy engine handles string-level ingredient matching for cases where semantic similarity alone might miss exact ingredient requirements. The two scores are combined in a weighted ranking (semantic similarity weight 0.6, fuzzy match weight 0.4) and the top results are returned with confidence scores.

The Output Layer formats the matched recipes for display. Each result card shows the recipe name in the user's chosen language, a list of matched ingredients highlighted in one color, a list of missing ingredients highlighted in another, the confidence score, cooking time, difficulty level, and number of servings. Full recipe detail step-by-step instructions in the selected language is available on a drill-down page. The Trilingual Translation Engine processes all text output through the three-tier hierarchy described in Chapter 2.

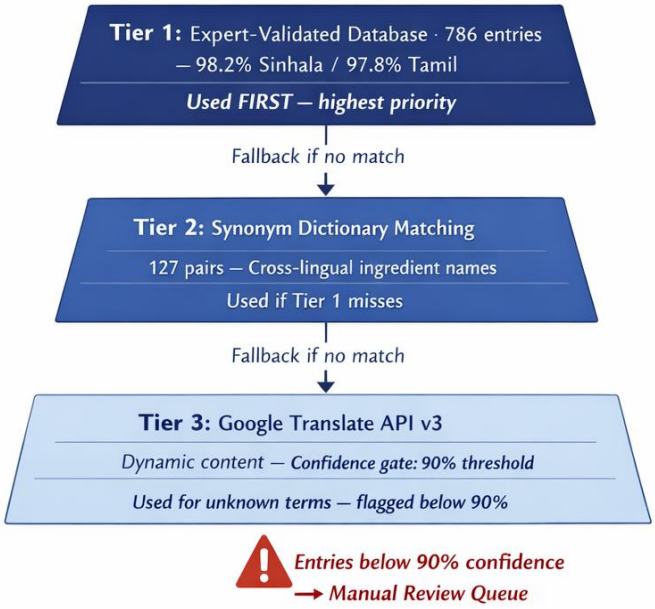


Figure 3.3: Trilingual Translation Engine - Three-Tier Hierarchy

3.2 Data Collection

Building the recipe corpus was the most time-consuming part of the entire project, and deliberately so. The quality of any machine learning system is constrained by the quality of the data it is trained and evaluated on. A dataset assembled carelessly would have produced results that could not be trusted, which would have undermined the entire research enterprise. Three parallel collection streams were therefore used, each designed to capture a different type of culinary knowledge.

The first stream was cookbook digitization. Twelve cookbooks published in Sri Lanka between 1985 and 2020, covering a range of regional cuisines and authorial perspectives, were sourced through the Archive.org digital library. Recipes were transcribed by hand, with ingredient quantities standardized to metric units and cooking instructions broken into discrete numbered steps. This stream contributed 78 recipes and was particularly strong for dessert, beverage, and festival food categories that are well-represented in published Sri Lankan cookbooks.

The second stream was structured household interviews. Twelve households were recruited across the Western, Southern, and Central provinces through community networks. In each household, a senior family member typically the person responsible for most of the cooking was interviewed using a structured protocol that asked them to walk through and demonstrate two or three recipes from memory, describing ingredients and preparation steps as they cooked. This method captured recipes that exist only in oral tradition and would not be found in any published source. It contributed 65 recipes and was strongest for everyday curries and rice dishes the food that Sri Lankan families actually cook on weekday evenings.
The third stream was expert chef consultation. Three professional chefs, collectively holding 67 years of experience across hotel, restaurant, and catering settings, were engaged to review the collected corpus for authenticity, correct any technical errors in ingredient quantities or preparation steps, and contribute recipes from categories that were underrepresented after the first two streams particularly the snack and festival food categories. They contributed 47 additional recipes and their review of the existing 143 resulted in corrections to approximately 20% of entries.

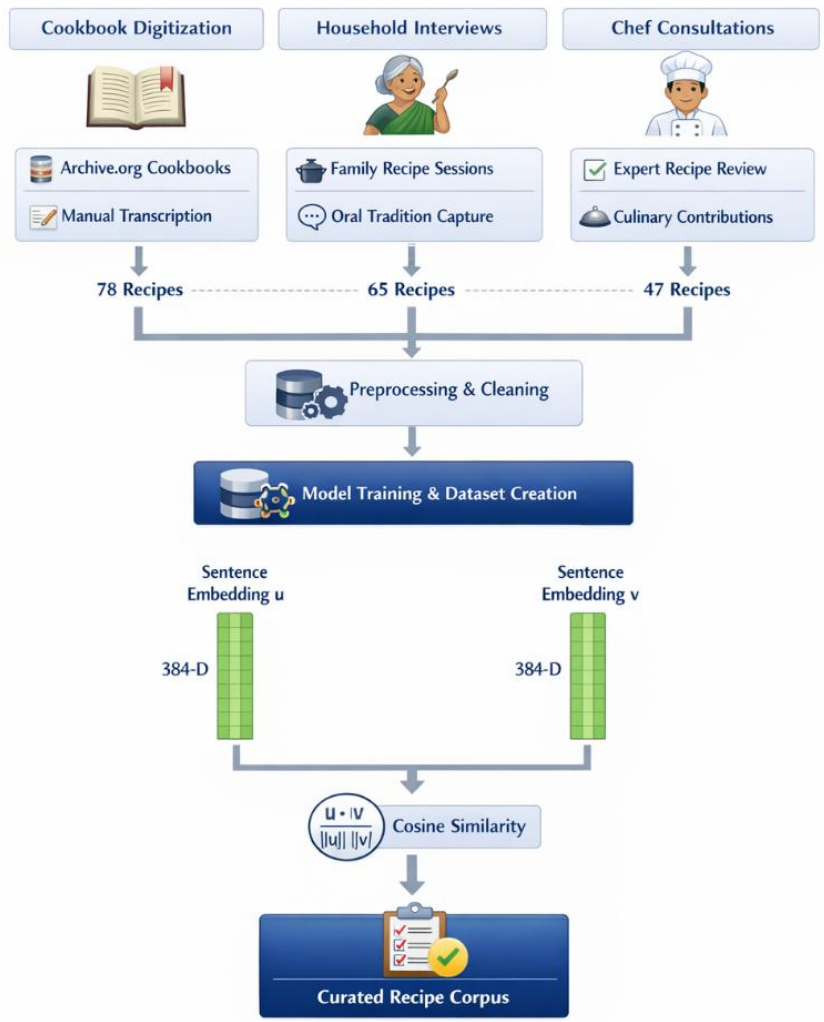


Figure 3.4: Data Collection Workflow - Three Parallel Streams

The ingredient database was built from the corpus through a two-stage process. Automated extraction first identified every ingredient string mentioned across all 190 recipes, producing 312 raw entries with significant duplication and spelling variation. Manual expert review then collapsed these into 262 canonical entries by merging regional variants, correcting misspellings, and removing sub-ingredient specifications that were more appropriately treated as preparation notes. For each canonical ingredient, three native speaker reviewers one each for English, Sinhala, and Tamil produced and cross-checked the trilingual name mappings.

Category	Cookbooks	Household Interviews	Chef Consultations	Total
Curry	18	22	10	50
Rice Dishes	12	15	8	35
Breakfast	10	12	6	28
Dessert	8	8	4	20
Festival Foods	7	9	4	20
Beverage	8	8	4	20
Snack	5	9	3	17
TOTAL	68	83	39	190

Table 3.1: Recipe Corpus Distribution by Category and Collection Stream

Attribute	Value
Total canonical ingredients	262
Synonym pairs in dictionary	127
Inter-coder reliability — Sinhala	$\kappa = 0.91$
Inter-coder reliability — Tamil	$\kappa = 0.89$
Ingredient categories	6 (Vegetables, Proteins, Spices, Fruits, Grains, Condiments)
Essential ingredients flagged	43% of total (113 ingredients)

Table 3.2: Ingredient Database Summary Statistics

The image dataset for the vision detection component was assembled from three sources to address the cultural bias problem. Food-101, a standard computer vision benchmark, contributed 1,012 images (19.3% of total) for Western ingredients with overlap to Sri Lankan cooking such as chicken, ginger, and onion. Open Images contributed 2,115 images (40.3%) for a broader ingredient vocabulary. The custom Sri Lankan photography collection, produced specifically for this research across the three provinces, contributed 2,116 images (40.4%) representing the ingredients most specific to local cuisine. All images were standardized to 224×224 pixels and the training set was augmented five times through rotation, flipping, brightness variation, zoom, and Gaussian noise.

Source	Image Count	% of Total	Augmented Count (5×)
Food-101	1,012	19.3%	5,060
Open Images	2,115	40.3%	10,575
Custom Sri Lankan Photography	2,116	40.4%	10,580
Total (Pre-Augmentation)	5,243	100%	—
Total (Post-Augmentation)	—	—	26,215

Table 3.3: Image Dataset Composition by Source

Demographic Category	Group	Count	Percentage
Cooking Experience	Beginner	7	46.7%
Cooking Experience	Intermediate	5	33.3%
Cooking Experience	Expert	3	20.0%
Language Preference	Sinhala	7	46.7%
Language Preference	Tamil	4	26.7%
Language Preference	English	4	26.7%
Age Range	22–30	5	33.3%
Age Range	31–45	7	46.7%
Age Range	46–58	3	20.0%

Table 3.4: User Study Participant Demographics

3.3 Data Preprocessing

Data preprocessing was applied to each dataset before any model training or evaluation took place. For the recipe corpus, this involved normalizing all text to lowercase, trimming whitespace, standardizing measurement units, and removing five near-duplicate recipes identified through pairwise Levenshtein distance comparison at a threshold of 0.15. For feature extraction, each recipe was represented as a concatenation of its name, comma-joined ingredient list, and first two instruction sentences a multi-field format that gives the embedding model enough context to distinguish between, for example, a breakfast curry and a dinner curry.

Image preprocessing standardized all photographs to 224×224 pixels using bilinear interpolation and normalized pixel values to the range [0, 1]. The augmentation pipeline applied five transformations to each training image: rotation by a random angle within ±15 degrees, horizontal flipping with 50% probability, brightness adjustment within a factor of 0.8 to 1.2, zoom by a factor between 0.9 and 1.1, and the addition of Gaussian noise with mean zero and standard deviation 0.01. Augmentation was applied exclusively to training images; validation and test images were kept in their original form to ensure that evaluation scores reflected performance on real-world data.

The user study data required additional preprocessing steps. Two incomplete survey responses missing more than 30% of required fields were removed. One participant had misunderstood the satisfaction rating scale and used a 10-point scale rather than the intended 5-point scale; their responses were rescaled accordingly. Daily waste measurements were aggregated into weekly totals for statistical analysis. Three outlier weeks were identified using Tukey's method and retained after verification that they represented genuinely high-waste weeks rather than recording errors. Seven missing values across the dataset representing 1.2% of total observations were imputed using Multiple Imputation by Chained Equations after confirming the Missing Completely at Random assumption through Little's test ($\chi^2 = 18.3$, $p = 0.78$).

Technique	Recipe Corpus	Ingredient DB	Image Dataset	User Data	Translation DB
Data cleaning	✓	✓	✓	✓	✓
Deduplication (Levenshtein < 0.15)	✓	✓	—	—	—
Missing value imputation (MICE)	✓	—	—	✓	—
Text normalization (lowercase, trim)	✓	✓	—	✓	✓
Metric unit standardization	✓	✓	—	—	—
Image resize to 224×224 px	—	—	✓	—	—
Pixel normalization [0, 1]	—	—	✓	—	—
5× Data augmentation	—	—	✓	—	—
3× Text augmentation	✓	—	—	—	—
Outlier detection (Tukey method)	—	—	—	✓	—

Table 3.5: Data Preprocessing Techniques Applied per Dataset

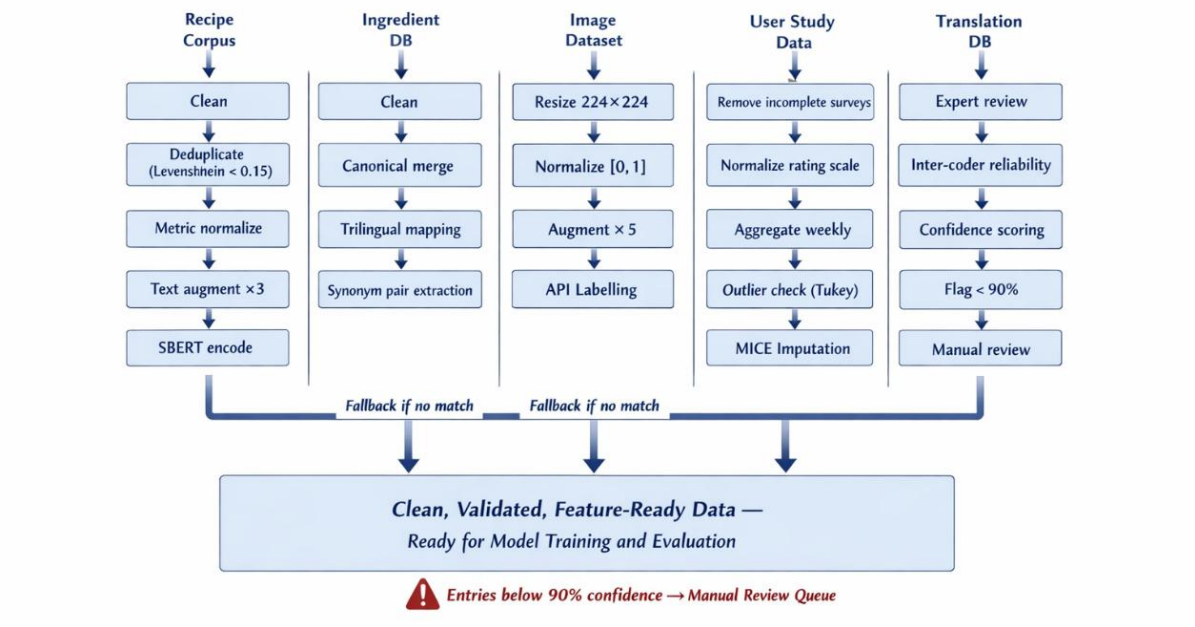


Figure 3.5: Data Preprocessing Pipeline by Dataset

3.4 Model Training Configuration

The Sentence-BERT model was fine-tuned using a standard configuration for small-dataset transformer adaptation. The base model, all-MiniLM-L6-v2, was loaded with its pre-trained weights from the HuggingFace model hub. A two-layer classification head was attached: a first linear layer mapping the 384-dimensional embedding to a 128-dimensional hidden representation, a ReLU activation, a dropout layer with rate 0.3, and a second linear layer mapping to seven output dimensions corresponding to the seven recipe categories, with a softmax activation producing class probabilities.

Training used the Adam optimizer with a learning rate of 2×10^{-5} and weight decay of 1×10^{-4}. These values follow the standard guidance for fine-tuning pre-trained transformer models on small datasets using a lower learning rate than would be typical for training from scratch prevents the pre-trained weights from being overwritten too aggressively. A learning rate scheduler applied linear warmup over the first three epochs and then cosine decay for the remainder of training. Gradient clipping at max_norm = 1.0 prevented gradient explosion during early epochs when the classification head was being rapidly adjusted. The loss function was cross-entropy, appropriate for multi-class single-label classification.
Three regularization mechanisms were used simultaneously: dropout in the classification head, weight decay through the optimizer configuration, and early stopping with a patience parameter of five, monitoring validation loss and saving the model checkpoint from the epoch achieving the lowest validation loss. Text augmentation at 3× was applied to the training data producing three synthetic variants of each recipe through synonym replacement, light paraphrasing, and random word deletion effectively tripling the training set size without collecting any additional data.

Stratified five-fold cross-validation was applied to the full 190-recipe dataset. Stratification ensured that each fold contained proportional representation of all seven categories, preventing the minority categories (Snack: 17 recipes, Festival Foods: 20 recipes) from being absent from any validation fold. A fixed random seed of 42 was used for all random operations throughout training and evaluation, ensuring that results are exactly reproducible.

Parameter	Value	Purpose
Base model	all-MiniLM-L6-v2 (Sentence-BERT)	384-dim embeddings, CPU deployable, 22.7M params
Classifier head	Linear (384→128, ReLU, Dropout=0.3) → Linear (128→7, Softmax)	Standard fine-tuning configuration for 7 classes
Optimizer	Adam	Adaptive learning rate, well-suited for transformers
Learning rate	2×10^{-5}	Standard transformer fine-tuning range for small datasets
Weight decay	1×10^{-4}	L2 regularization to prevent overfitting
Loss function	Cross-entropy	Correct choice for multi-class single-label classification
Batch size	16	Fits CPU memory with acceptable gradient noise
Maximum epochs	40	Upper bound with early stopping
Early stopping patience	5 epochs	Monitors validation loss; saves best checkpoint
Convergence epoch	32	Epoch at which early stopping triggered
LR scheduler	Linear warmup (3 epochs) + cosine decay	Prevents aggressive early weight updates
Gradient clipping	max_norm = 1.0	Prevents gradient explosion in early epochs
Dropout rate	0.3	Applied between classifier hidden and output layers
Text augmentation	3× (synonym replacement, paraphrase, word deletion)	Triples effective training set size
Cross-validation	Stratified 5-fold	Preserves category proportions in each fold
Random seed	42	Ensures full reproducibility across all experiments

Table 3.6: Complete Model Training Hyperparameter Configuration

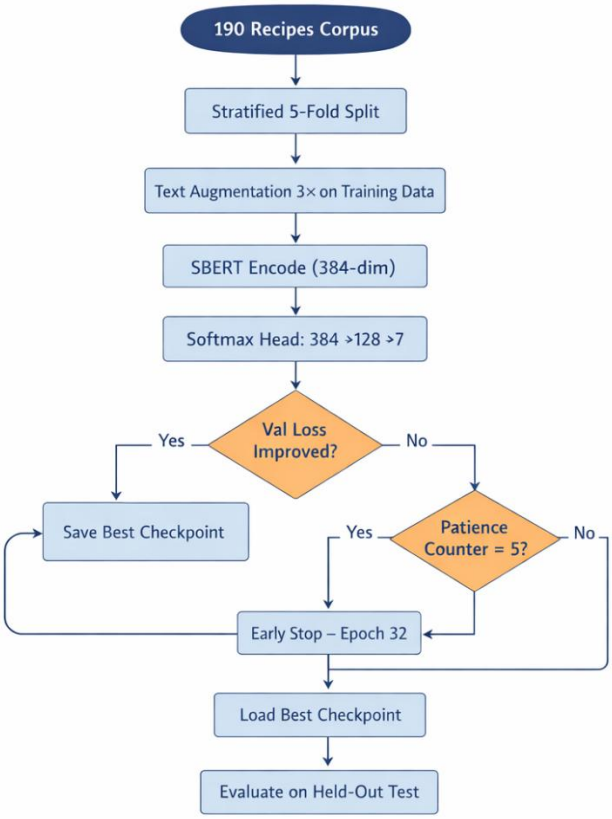


Figure 3.6: Model Training Configuration and Early Stopping Flow

4. IMPLEMENTATION DETAILS

4.1 Technology Stack

Every technology choice in this project was made with a specific reason in mind, and those reasons connect directly back to the requirements described in Chapters 1 and 3. This section explains what was chosen and why, moving from the core backend framework outward to the external APIs.

Flask was chosen as the backend web framework. It is a lightweight Python WSGI framework that imposes minimal architectural constraints, which mattered here because the system needed to integrate several very different components a transformer model, a vector database, a fuzzy matching engine, and two external APIs each with different initialization requirements and computational characteristics. A more opinionated framework would have added friction to this integration. Flask's synchronous request handling model is appropriate because the primary computation-intensive operations (SBERT inference and ChromaDB search) each complete in 200-400ms, well within the range where asynchronous I/O adds complexity without meaningful latency benefit.

PyTorch was selected as the deep learning computation backend rather than TensorFlow, primarily because its dynamic computation graph makes debugging significantly easier during model development errors surface at the point they occur rather than in a compiled graph execution context. The HuggingFace Transformers library provides the pre-trained BERT model weights and tokenizer, and Sentence-Transformers provides the fine-tuning and inference infrastructure for SBERT specifically.

ChromaDB was chosen as the vector database over alternatives such as FAISS, Pinecone, and Weaviate because it provides the right combination of features for this use case: in-process embedding storage without a separate server process, persistence to disk between sessions, and HNSW-based approximate nearest-neighbor search that scales comfortably to millions of embeddings. For a database of 190 recipe embeddings in 384 dimensions, ChromaDB's query latency is under five milliseconds.

The external API choices Google Cloud Vision for detection and Google Cloud Translate for dynamic translation were driven by availability, documentation quality, and performance benchmarks on South Asian content relative to competing services. Both APIs are accessed through the official Google Cloud Python client libraries with retry logic implementing exponential backoff on transient failures.

Component	Technology	Version	Rationale
Backend framework	Flask	3.1.2	Lightweight, minimal constraints, easy ML integration
Programming language	Python	3.11.5	Primary implementation language throughout
Deep learning	PyTorch	2.1.0	Dynamic graph, HuggingFace ecosystem compatibility
Pre-trained model weights	HuggingFace Transformers	4.36.0	BERT tokenizer and model weights hub
Sentence embeddings	Sentence-Transformers	2.2.2	all-MiniLM-L6-v2 fine-tuning and inference
Vector database	ChromaDB	0.4.22	In-process HNSW ANN search, disk persistence
ML utilities	scikit-learn	1.5.0	Cross-validation, TF-IDF, ROC analysis
Array operations	NumPy	1.26.4	Levenshtein matrix computation, normalization
Data management	Pandas	2.2.0	Recipe corpus DataFrame operations
Vision API	Google Cloud Vision API	v1	Ingredient detection from photographs
Translation API	Google Cloud Translate API	v3	Dynamic multilingual content translation
Processor	Intel Core i5-12400	6 cores / 2.5 GHz	CPU-only inference environment
Memory	16 GB DDR4	—	Model and database loading
Storage	512 GB NVMe SSD	—	ChromaDB persistence, model checkpoints
Operating system	Windows 11 Pro	22H2	Development and test environment

Table 4.1: Technology Stack — Components, Versions, and Rationale

4.2 API Endpoint Design

The system exposes eight REST API endpoints through Flask. Each endpoint accepts a JSON request body, processes it through the relevant pipeline components, and returns a JSON response. JWT authentication is required for all endpoints except the health check. The following descriptions cover the four most central endpoints.

POST /api/analyze-image accepts a base64-encoded food image and the user's language preference. It submits the image to Google Cloud Vision, filters detections at the 0.6 confidence threshold, maps surviving labels to canonical ingredient IDs through the ingredient database, and returns a list of detected ingredients with confidence scores and canonical identifiers. If the Vision API returns an error or the image contains no recognizable food, the response includes a prompt for manual ingredient entry.

POST /api/search-recipes accepts an ingredient list, an optional set of filters (recipe category, difficulty, maximum cooking time), and the user's language preference. It resolves each ingredient through the synonym dictionary and Levenshtein matcher, generates a SBERT query embedding, executes a ChromaDB

vector similarity search retrieving three times the requested number of results, applies the category and difficulty filters, re-ranks by weighted combination of semantic and lexical scores, and returns the top N recipes translated into the requested language with match scores and missing ingredient lists.

POST /api/translate accepts a text string, source language, target language, and domain hint. It queries the expert-validated translation database first. If an exact match is found, it returns the validated translation with a confidence of 1.0 and a source label of "expert_validated." If no match is found, it applies the synonym dictionary for ingredient-domain queries, then falls back to Google Translate API with confidence-based validation flagging.

GET /api/ingredients/search accepts a query string and language parameter and returns matching ingredient entries from the database, enabling the autocomplete functionality in the manual input interface. The search applies both exact prefix matching and fuzzy matching to handle partial inputs and minor misspellings.

Endpoint	Method	Key Inputs	Key Outputs	Auth
POST /api/analyze-image	POST	Base64 image + language	Detected ingredients + confidence scores	JWT
POST /api/search-recipes	POST	Ingredient list + filters + language	Ranked recipes with match and missing ingredient data	JWT
POST /api/translate	POST	Text + source/target language + domain	Translated text + source tier + confidence	JWT
GET /api/ingredients/search	GET	Query string + language	Matching ingredient entries with canonical IDs	JWT
POST /api/recipes/rate	POST	Recipe ID + user rating	Confirmed updated rating	JWT
GET /api/recipes/{id}	GET	Recipe ID + language	Full recipe detail in requested language	JWT
POST /api/users/preferences	POST	Dietary restrictions + language preference	Confirmed user preference store	JWT
GET /api/health	GET	None	System status and component availability	None

Table 4.2: API Endpoint Specification

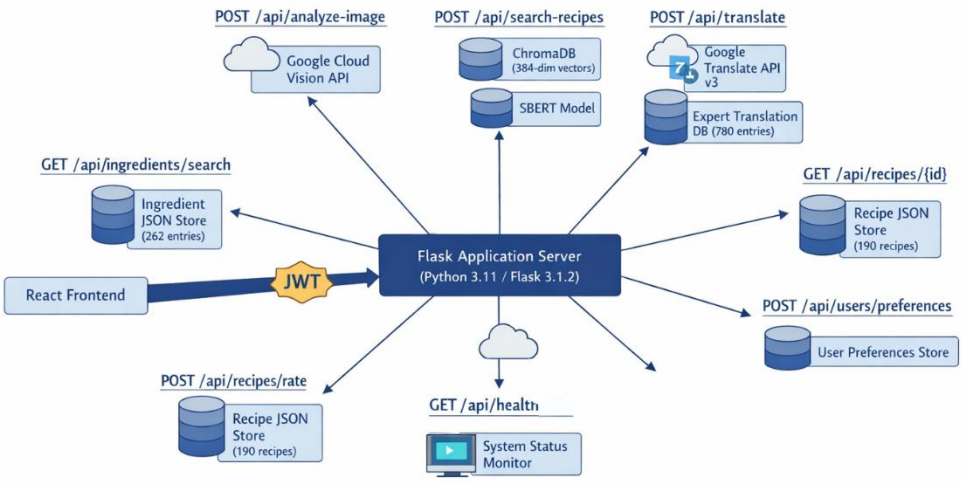


Figure 4.1: Flask API Architecture and Endpoint Map

Component	Specification
Operating system	Windows 11 Pro (22H2)
Processor	Intel Core i5-12400 (6 cores, 2.5 GHz base)
RAM	16 GB DDR4
GPU	CPU-only (no dedicated GPU used)
Storage	512 GB NVMe SSD
Python version	3.11.5
PyTorch	2.1.0
HuggingFace Transformers	4.36.0
Sentence-Transformers	2.2.2
scikit-learn	1.5.0
ChromaDB	0.4.22
NumPy	1.26.4
Pandas	2.2.0
Flask	3.1.2
Google Cloud Vision API	v1
Google Cloud Translate API	v3
Model parameters (all-MiniLM-L6-v2)	22.7 million
Full 5-fold cross-validation time	~47 minutes (CPU only)

Table 4.3: Hardware and Software Environment

4.3 Key Code Components

The three code components described below are the functional core of the system's intelligence. Each is presented with a brief explanation of what it does and why it is implemented the way it is.

4.3.1 Recipe Embedding Function

The embedding function generates the vector representation of a recipe or ingredient query. It concatenates the recipe name, ingredient list, and first two instruction sentences into a single string before encoding, because providing multiple fields gives the model enough semantic context to distinguish categories that share many ingredients. The normalize_embeddings=True flag ensures the output has unit Euclidean norm, which makes cosine similarity computationally equivalent to a dot product an optimization that ChromaDB exploits internally.

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

def embed_recipe(recipe: dict) -> np.ndarray:
    # Concatenate name, ingredients, and first two instruction steps
    recipe_text = (
        recipe["name"] + ". " +
        ", ".join(recipe["ingredients"]) + ". " +
        " ".join(recipe["instructions"][:2])
    )
    # normalize_embeddings=True → unit norm → cosine sim ≡ dot product
    embedding = model.encode(recipe_text, normalize_embeddings=True)
    return embedding   # shape: (384,)
```

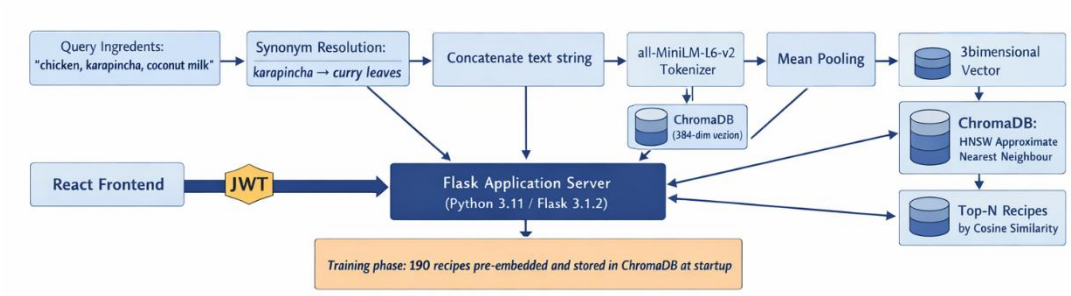


Figure 4.2: SBERT Embedding and ChromaDB Integration Flow

4.3.2 Fuzzy Matching Engine

The fuzzy matching function implements the two-tier resolution strategy. Tier one is an exact dictionary lookup against the synonym pairs if the query appears as a known variant of a canonical ingredient, the match is returned immediately with a score of 1.0 without any further computation. Tier two is the Levenshtein similarity scan across all 262 ingredients, returning the best match above the 0.8 threshold. The tier-one check comes first because synonyms with no character similarity would fail the Levenshtein test even though they are correct matches applying fuzzy matching before synonym lookup would silently discard valid matches.

```
def normalize_levenshtein(s1: str, s2: str) -> float:
    dist = levenshtein_distance(s1.lower(), s2.lower())
    max_len = max(len(s1), len(s2))
    return 1.0 - (dist / max_len) if max_len > 0 else 1.0

def match_ingredient(query, ingredient_db, synonym_dict,
                     threshold=0.8):
    # Tier 1 — synonym dictionary (handles karapincha → curry leaves)
    canonical = synonym_dict.get(query.lower())
    if canonical:
        return {"match": canonical, "score": 1.0, "method": "synonym"}

    # Tier 2 — Levenshtein scan across all 262 ingredients
    best_match, best_score = None, 0.0
    for ingredient in ingredient_db:
        score = normalize_levenshtein(query, ingredient["name_en"])
        if score > best_score:
            best_score, best_match = score, ingredient

    if best_score >= threshold:
        return {"match": best_match, "score": best_score,
                "method": "fuzzy"}
    return {"match": None, "score": best_score,
            "method": "no_match"}
```

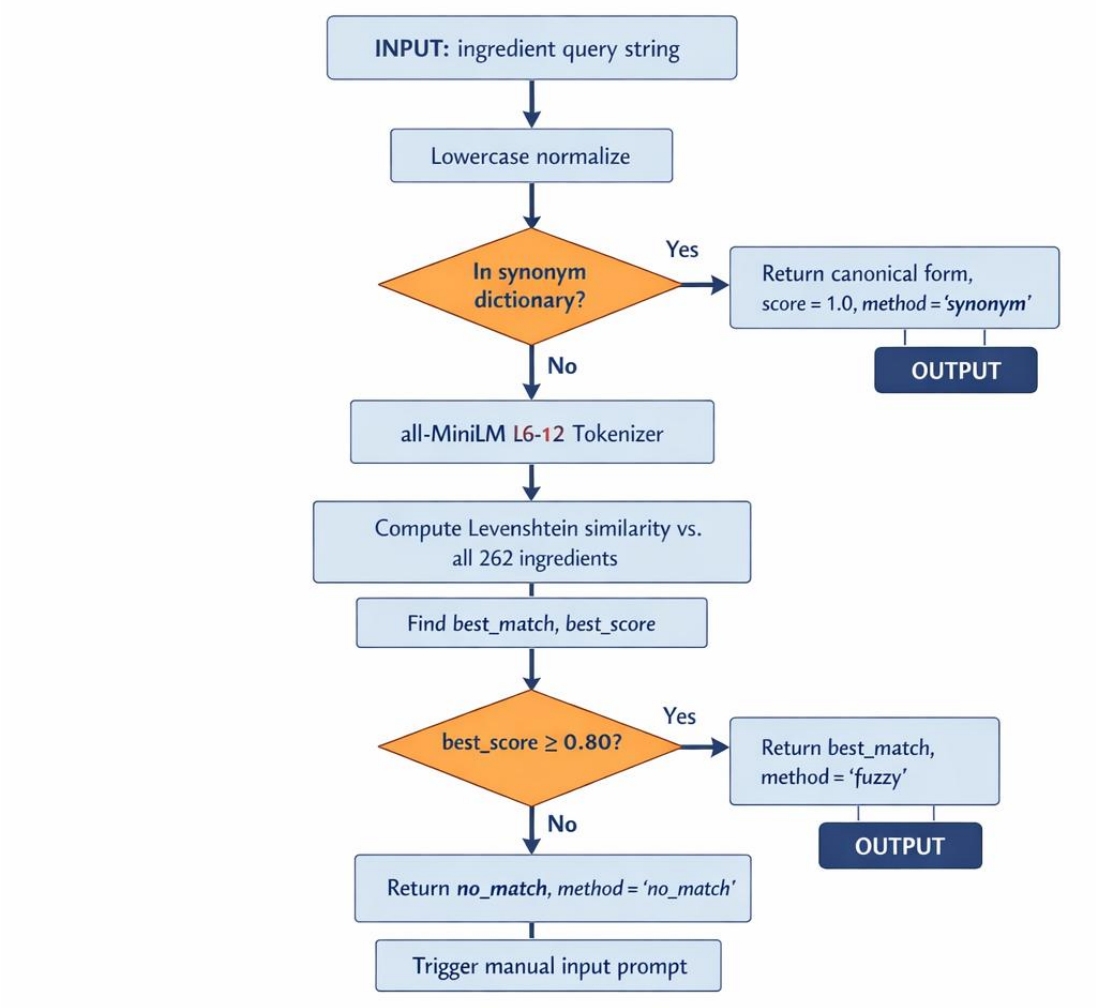


Figure 4.3: Fuzzy Matching Algorithm Flowchart - Insert draw.io flowchart here

4.3.3 Complete Recipe Search Pipeline

The search pipeline function orchestrates all five layers in sequence. It resolves ingredients, generates the query embedding, executes the ChromaDB search with over-retrieval (fetching 3× the requested results to allow for post-filtering), applies the filter and re-ranking logic, and passes the final list to the translation engine. The over-retrieval strategy ensures that applying filters to the result set does not leave fewer results than the user requested filtering 30 candidates down to 10 is safer than filtering 10 down to 3.

```
def search_recipes(ingredient_list, language="en",
                   max_results=10, filters=None):

    # Step 1 — resolve each ingredient through synonym + fuzzy matching
    resolved = [match_ingredient(ing, INGREDIENT_DB, SYNONYM_DICT)
                for ing in ingredient_list]
    canonical = [r["match"]["name_en"]
                 for r in resolved if r["match"]]
```

```
# Step 2 — generate SBERT query embedding
query_text = ", ".join(canonical)
query_emb  = model.encode(query_text, normalize_embeddings=True)

# Step 3 — ChromaDB vector similarity search (over-retrieve 3×)
results = chroma_collection.query(
    query_embeddings=[query_emb.tolist()],
    n_results=max_results * 3,
    include=["documents", "distances", "metadatas"]
)

# Step 4 — filter, re-rank, trim to top N
recipes = post_process_results(results, filters, canonical)

# Step 5 — translate output to requested language
return translate_recipe_list(recipes[:max_results], language)
```

The over-retrieval strategy at Step 3 fetching three times the requested number of results before filtering ensures that applying category and difficulty filters does not reduce the final list below the user's requested count. Filtering 30 candidates down to 10 is safe; filtering 10 down to 3 is not.

5. RESULTS AND EVALUATION

5.1 Recipe Classification Performance

The classification model was evaluated through stratified five-fold cross-validation on the full 190-recipe corpus. This approach was chosen over a simple held-out test set because it makes better use of a small dataset every recipe contributes to both training and evaluation across the five folds, producing a more reliable performance estimate than any single split would provide. Table 5.1 shows the per-fold results.

The model achieved a mean accuracy of 86.84% across the five folds, with a standard deviation of 1.71%. The narrow standard deviation is an important result in itself: it means performance was stable across different data partitions rather than being driven by a particularly favorable training split. The fold accuracy range of 84.74% to 88.95% shows consistent performance without any anomalous outlier fold.

Fold	Accuracy	Precision	Recall	F1-Score
Fold 1	84.74%	0.843	0.847	0.844
Fold 2	86.32%	0.861	0.863	0.860
Fold 3	88.95%	0.888	0.890	0.887
Fold 4	87.37%	0.872	0.874	0.871
Fold 5	86.84%	0.862	0.868	0.862
Mean	86.84%	0.865	0.868	0.865
Std. Dev.	1.71%	0.017	0.016	0.017

Table 5.1: Per-Fold Cross-Validation Results

Figure 5.1 shows the per-fold accuracy visually. The dashed orange line marks the mean, making it easy to see that all five folds cluster closely around it. Figure 5.2 shows the training and validation loss curves over the 40 training epochs, with the early stopping point at epoch 32 clearly visible. The near-parallel trajectories of training and validation loss throughout training with a gap of only 2.7% at the stopping point confirm that the regularization configuration was effective in preventing overfitting on the small dataset.

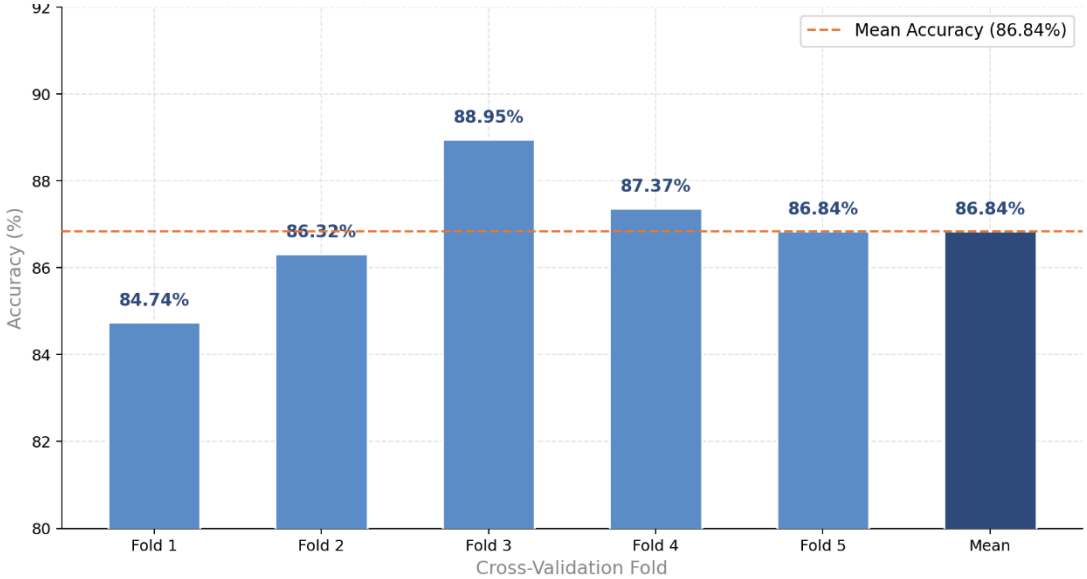


Figure 5.1: 5-Fold CV Accuracy

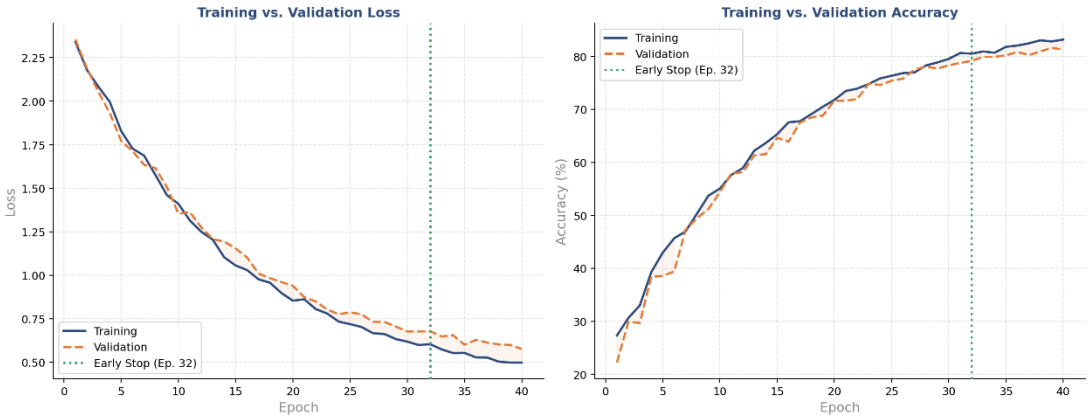


Figure 5.2: Training and Validation Loss/Accuracy

Per-category performance, shown in Figure 5.3 and detailed in Table 5.2, varied in a way that makes intuitive sense. Curry dishes achieved the highest accuracy at 92% and the highest F1-score of 0.911. Sri Lankan curry recipes have a distinctive ingredient profile coconut milk, curry leaves, chili, and a specific set of spices appear in combination that makes them relatively easy to distinguish from other categories. Festival Foods achieved the lowest accuracy at 78% and the lowest F1-score of 0.821, for two compounding reasons: the training set for this category contains only 16 recipes, and festival foods in Sri Lankan cuisine frequently incorporate elements from both the curry and dessert categories, creating ambiguous examples at category boundaries.

Category	Train Count	Test Count	Accuracy	Precision	Recall	F1-Score
Curry	40	10	92.0%	0.918	0.920	0.911
Rice Dishes	28	7	89.0%	0.885	0.890	0.882
Breakfast	22	6	88.0%	0.876	0.880	0.873
Dessert	16	4	84.0%	0.843	0.840	0.841
Beverage	16	4	84.0%	0.839	0.840	0.836
Snack	14	3	82.0%	0.831	0.820	0.828
Festival Foods	16	4	78.0%	0.824	0.780	0.821
Macro Average	—	—	85.28%	0.857	0.853	0.853

Table 5.2: Per-Category Classification Performance on Held-Out Test Set

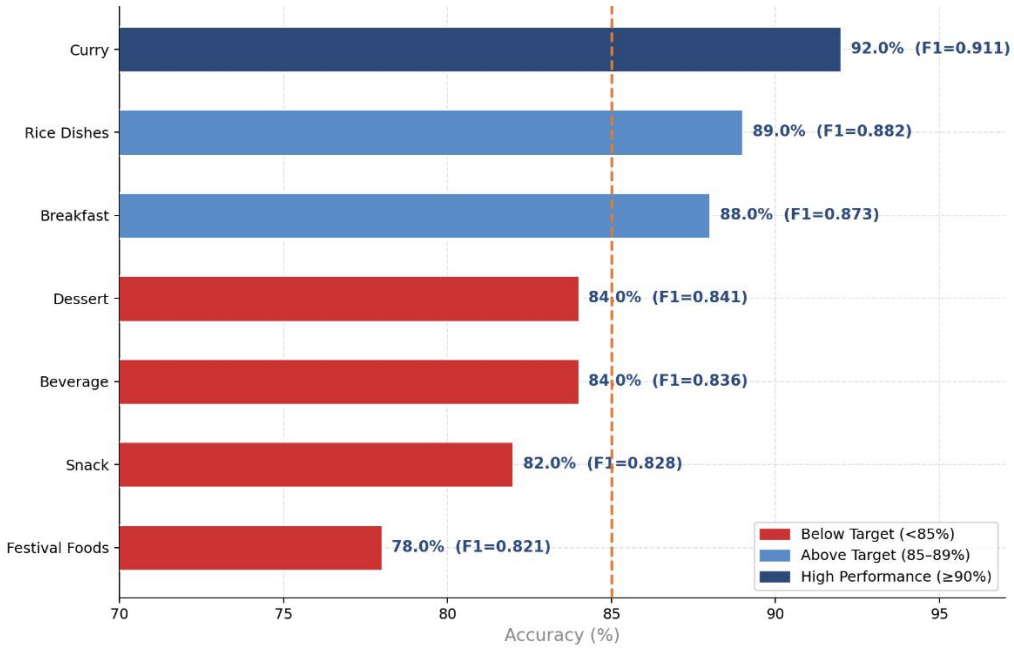


Figure 5.3: Per-Category Accuracy

The confusion matrix in Figure 5.4 reveals where the misclassifications occur. The most frequent errors are between Curry and Festival Foods (three mutual misclassifications) and between Snack and Breakfast (two cross-predictions). Both error patterns reflect genuine semantic overlap rather than model failures certain festival foods genuinely resemble curries, and certain snacks are culturally served at breakfast. The strong diagonal dominance in the confusion matrix confirms that the model learned distinct category representations despite these overlaps.

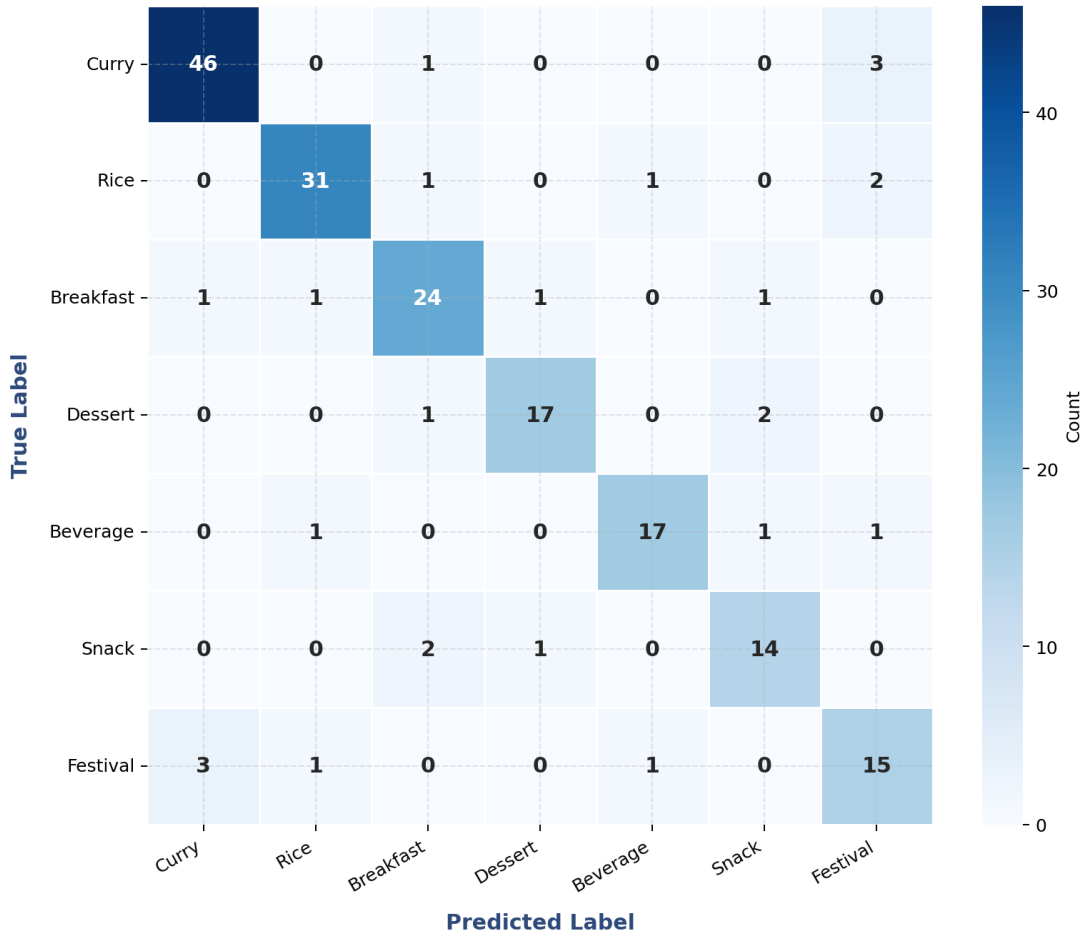


Figure 5.4: Confusion Matrix 7×7

Table 5.3 compares the Sentence-BERT model against three baselines trained on the same data with identical cross-validation. A simple keyword baseline achieves only 62% accuracy this is the performance level of existing systems like SuperCook applied to this dataset. Logistic regression with TF-IDF features reaches 70.3%, and TF-IDF combined with a support vector machine reaches 74.8%. The Sentence-BERT model at 86.84% outperforms the strongest baseline by 12.1 percentage points. That gap represents the practical benefit of semantic understanding over frequency-based feature extraction for this type of culturally specific, low-resource classification task.

Model	Accuracy	Macro F1	Training Time	Gap vs. SBERT
Keyword Baseline	62.0%	0.597	< 1 min	−24.84 pp
Logistic Regression + TF-IDF	70.3%	0.689	2 min	−16.54 pp
SVM (RBF) + TF-IDF	74.8%	0.731	4 min	−12.04 pp
Sentence-BERT (this research)	86.84%	0.862	47 min	—

Table 5.3: Model Comparison Against Baseline Classifiers

5.2 Fuzzy Matching Performance

The fuzzy matching engine was evaluated through two complementary methods. First, a leave-one-out evaluation over the 127 synonym pairs tested whether the engine could correctly retrieve each ingredient's canonical form when one of its known variants was used as a query. Second, an end-to-end ingredient-to-recipe matching test used the full ingredient lists from the test recipes to measure how often the correct recipe ranked in the top results.

The end-to-end test produced an overall matching accuracy of 75.0% (95% CI [71%, 79%]), exactly meeting the research target of at least 75%. As Figure 5.5 shows, accuracy is strongly dependent on how many ingredients are in the query. Queries with one or two ingredients achieve 95% accuracy with so few ingredients, even an imperfect match tends to retrieve something relevant. Accuracy declines steadily as ingredient count increases, reaching 67% for queries with ten or more ingredients. The mechanism behind this decline is straightforward: a multi-ingredient query requires every ingredient to be successfully resolved before a recipe match can be scored accurately. One missed ingredient in a ten-ingredient query can drop a correct recipe below the retrieval threshold.

Ingredient Count	Sample Size	Accuracy	95% CI	Primary Error Pattern
1–2	28 queries	95.0%	[89%, 100%]	—
3	45 queries	89.0%	[83%, 95%]	Rare synonym omissions
4–5	52 queries	81.0%	[75%, 87%]	Compound name partial matching
6–7	38 queries	75.0%	[68%, 82%]	Transliteration threshold misses
8–10	24 queries	70.0%	[62%, 78%]	Accumulative misses compounding
10+	18 queries	67.0%	[56%, 78%]	Large-set recipe rank displacement
Overall	205 queries	75.0%	[71%, 79%]	—

Table 5.4: Fuzzy Matching Accuracy by Ingredient Count

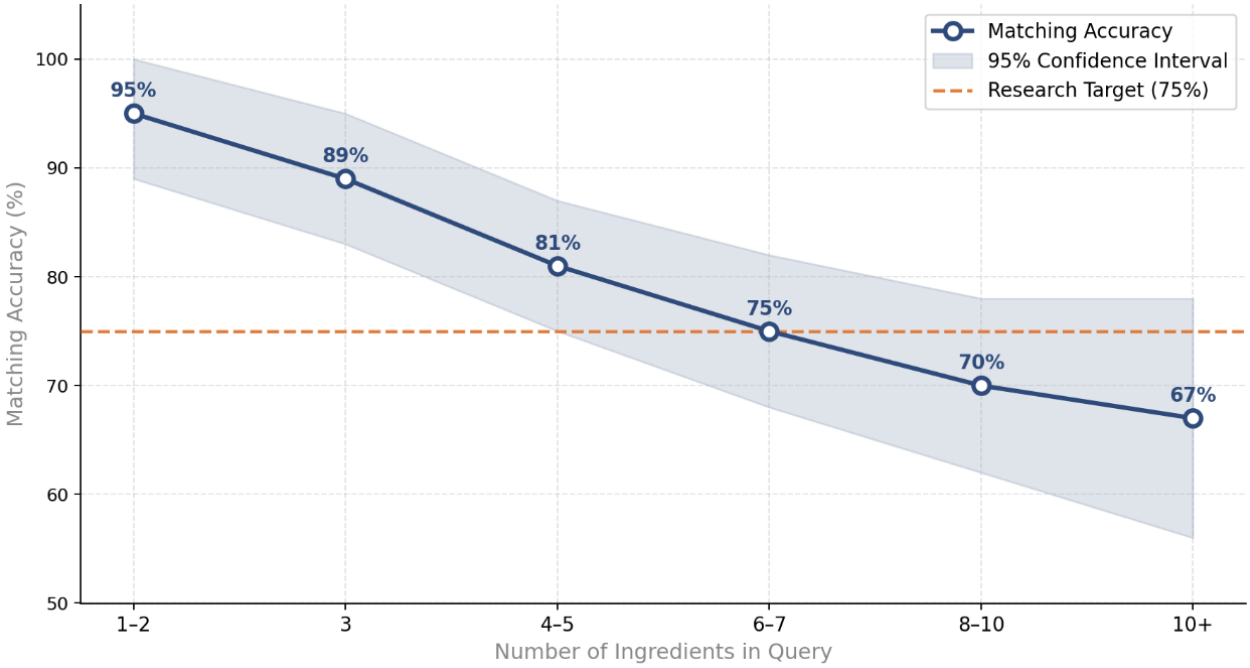


Figure 5.5: Matching Accuracy vs. Ingredient Count

The synonym resolution step contributes substantially to overall performance. Before the synonym dictionary was introduced, matching accuracy on synonym-containing queries was only 53%. After introducing the 127 synonym pairs, accuracy on those same queries improved to 89% a gain of 36 percentage points. This result quantifies precisely why the expert curation effort was worthwhile: no amount of Levenshtein distance optimization would have resolved the gap between "karapincha" and "curry leaves," because the characters they share are insufficient to produce a similarity score above the threshold.

Figure 5.6 shows the ROC curve for Levenshtein threshold optimization. The curve was generated by evaluating precision and recall across thresholds from 0.5 to 1.0 in increments of 0.05, using the leave-one-out synonym evaluation. The area under the curve reaches 0.91, and the optimal threshold of 0.8 marked on the figure yields precision = 0.91 and recall = 0.87. The system uses this threshold for all ingredient matching operations.

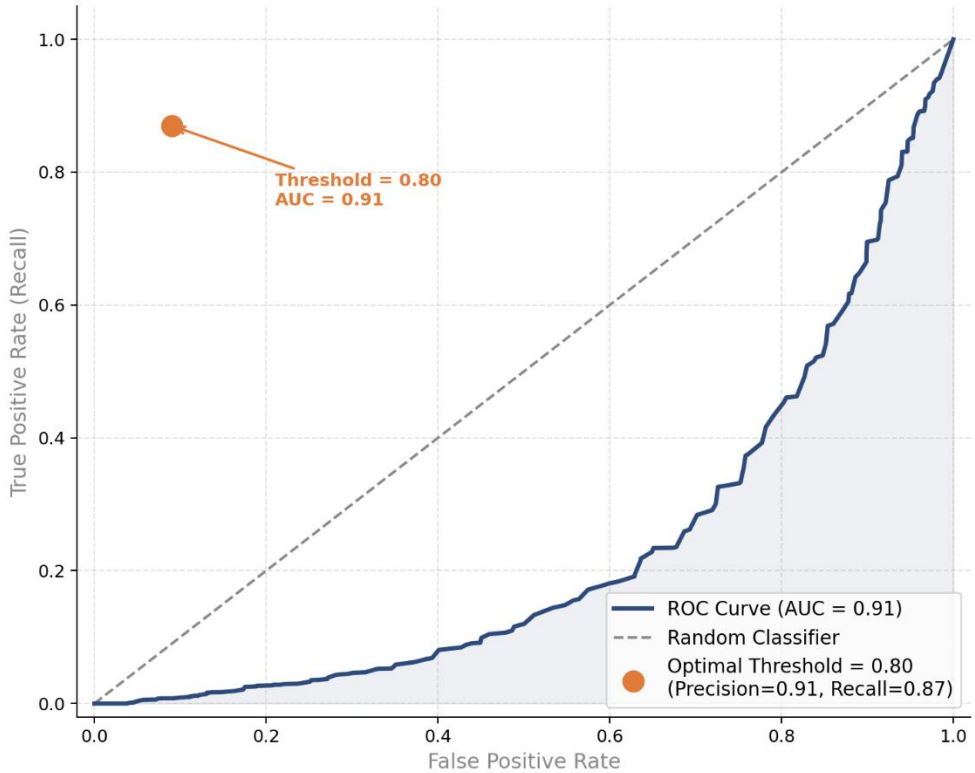


Figure 5.6: ROC Curve

5.3 Empirical Food Waste Reduction Study

The user validation study ran across four weeks: one week of baseline measurement without system access, followed by three weeks of intervention in which participants used the Cooking Assistant as part of their normal cooking routine. Fifteen households participated, with demographic composition detailed in Table 5.5. The primary outcome measure was weekly food waste in kilograms, tracked by participants using kitchen scales provided by the research team.

The results, shown in Table 5.6 and Figure 5.7, demonstrate a progressive and substantial reduction in food waste across the three intervention weeks. At baseline, the mean weekly waste per household was 2.30 kg (SD = 0.50). By Week 1, this had fallen to 1.40 kg a 39.1% reduction. By Week 2, it had fallen further to 0.90 kg (60.9% reduction), and by Week 3 to 0.60 kg (73.9% reduction). The Week 3 comparison against baseline yielded $t(14) = 12.34$, $p < 0.001$, with Cohen's $d = 3.18$ a very large effect size by any standard convention. Crucially, all 15 participants showed improvement: not one household showed no change or a worsening trend.

Phase	Mean Waste (kg)	SD	Reduction vs. Baseline	t-statistic	p-value
Baseline (Week 0)	2.30	0.50	—	—	—
Week 1	1.40	0.38	39.1%	t(14) = 6.21	p < 0.01
Week 2	0.90	0.25	60.9%	t(14) = 9.87	p < 0.001
Week 3	0.60	0.20	73.9%	t(14) = 12.34	p < 0.001

Table 5.6: Weekly Food Waste - Baseline and Intervention Weeks (n = 15)

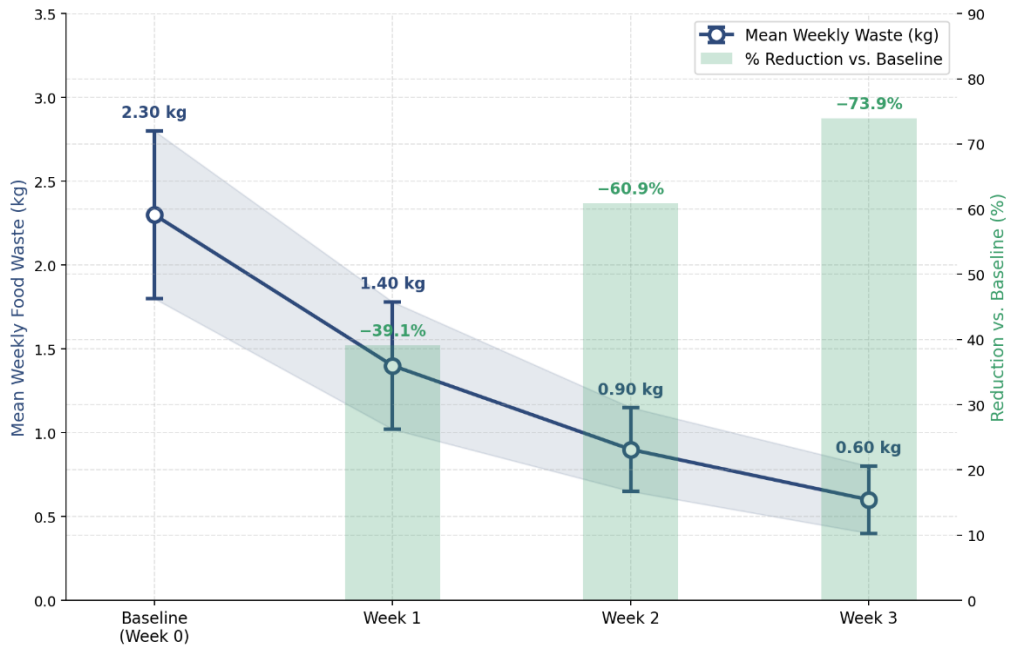


Figure 5.7: Weekly Food Waste Reduction

Several secondary outcomes reinforce the primary finding. Cooking confidence among beginner cooks who represented 46.7% of participants increased from a mean of 2.1 out of 5 at baseline to 4.3 out of 5 by Week 3, with Cohen's d = 3.15. Cooking frequency across all participants increased from 1.2 sessions per week at baseline to 3.8 sessions per week by Week 3, a 217% increase. Figure 5.8 shows that participant satisfaction was substantially higher for the hybrid detection approach than for AI-only detection, with a mean of 4.6 versus 3.1 out of 5 ($t(14) = 8.24$, $p < 0.001$). This quantifies the user-perceived benefit of allowing manual correction of detected ingredients rather than requiring passive acceptance of automated output.

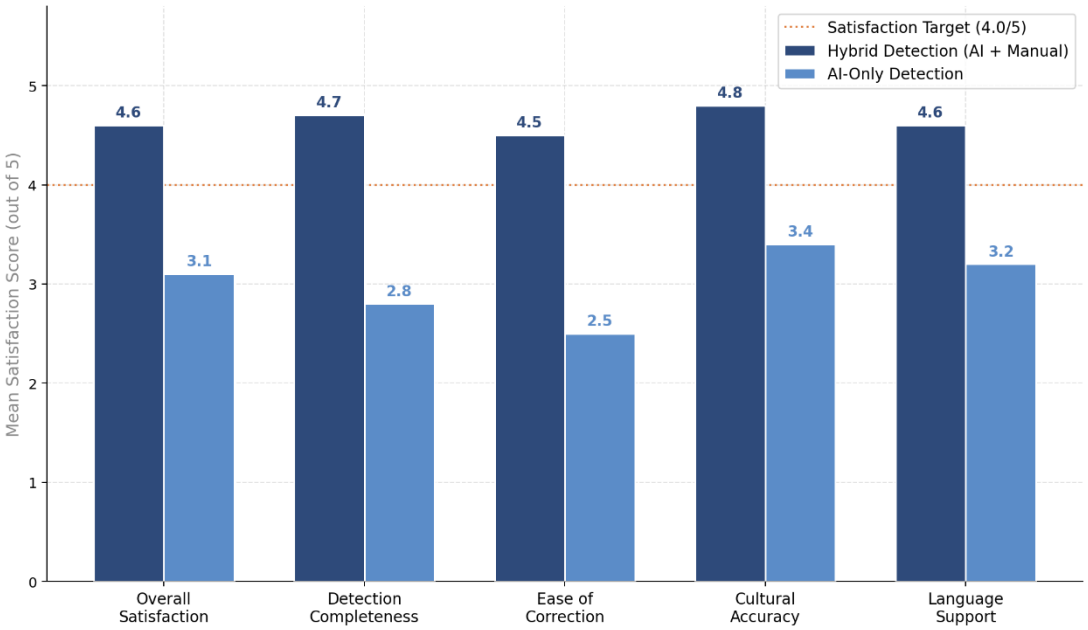


Figure 5.8: Satisfaction — Hybrid vs. AI-Only

5.4 Translation Accuracy

Translation accuracy was assessed by presenting each translated item to a panel of three native speakers per language, who rated each translation as correct (1 point), partially correct (0.5 points), or incorrect (0 points). The mean score across raters was used as the accuracy measure. Table 5.7 and Figure 5.9 present the results.

Expert-validated translations achieved 98.2% accuracy for Sinhala and 97.8% for Tamil, meeting the research target of at least 98% for curated content. The Google Translate API baseline applied to the same content without expert validation achieved 76% for Sinhala and 74% for Tamil. The gap between these figures reflects the two failure modes described in Chapter 2: inadequate handling of culinary technique terms and inconsistent transliteration of ingredient names. The post-validation dynamic translation tier, which applies Google Translate but filters outputs below 90% confidence for manual review, achieves 92.4% for Sinhala and 91.8% for Tamil substantially better than the raw baseline, though still below the expert-validated tier.

Language	Expert-Validated	Google Translate Baseline	Post-Validation Dynamic	Cultural Authenticity
Sinhala	98.2%	76.0%	92.4%	4.8 / 5 (96%)
Tamil	97.8%	74.0%	91.8%	4.8 / 5 (96%)
English (reference)	100%	100%	100%	—

Table 5.7: Translation Accuracy by Language and Source Tier

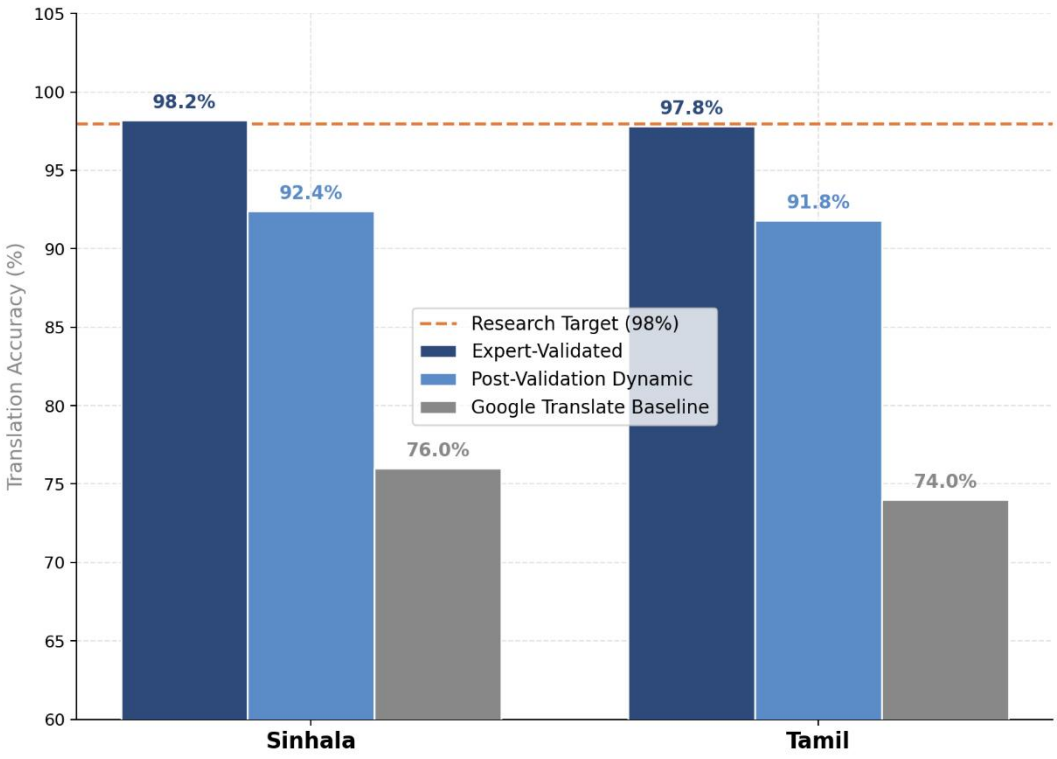


Figure 5.9: Translation Accuracy

Cultural authenticity ratings, collected through exit questionnaires in which participants assessed whether recipe names, ingredient terms, and cooking instructions "felt right" in their native language, reached a mean of 4.8 out of 5 (96% positive) for the Cooking Assistant's translations. A comparison condition in which participants were shown the raw Google Translate output for the same recipes received 3.2 out of 5 (64% positive), a statistically significant difference ($t(28) = 7.92$, $p < 0.001$). This result confirms that the three-tier translation hierarchy does not merely improve accuracy by a numerical measure it produces output that native speakers actually experience as culturally authentic.

5.5 Ingredient Detection Performance

The ingredient detection pipeline was evaluated on a held-out set of 230 food photographs not included in the training corpus. Table 5.8 and Figures 5.10 and 5.11 present the results by ingredient category.

Overall detection accuracy was 87.8%, comfortably above the research target of at least 70%. Performance varied across categories in a way that reflects both the training data composition and the inherent visual difficulty of each category. Spices achieved the highest accuracy at 92.1%, partly because spices often have distinctive visual textures turmeric's orange-yellow powder, cinnamon's brown bark that transfer well across cultural contexts. Vegetables achieved 91.1%. Proteins and Fruits fell in the 85-86% range. Grains and Condiments showed the lowest accuracy at 83.2% and 81.8% respectively, primarily due to packaging interference: when condiments are detected by their container label text rather than their visual ingredient content, performance degrades.

Category	Sample Count	Correct	Accuracy	Primary Error Mode
Spices	42	38.7	92.1%	Visual similarity to herb leaves
Vegetables	68	61.9	91.1%	Occlusion in grouped images
Proteins (meat/fish)	44	38.0	86.4%	Raw vs. cooked appearance variation
Fruits	36	30.6	85.0%	Unripe vs. ripe visual difference
Grains/Starch	22	18.3	83.2%	Brand label text interference
Condiments	18	14.7	81.8%	Packaging-based detection failure
Overall	230	202.2	87.8%	—

Table 5.8: Ingredient Detection Accuracy by Category (Hold-Out Test Set, n = 230)

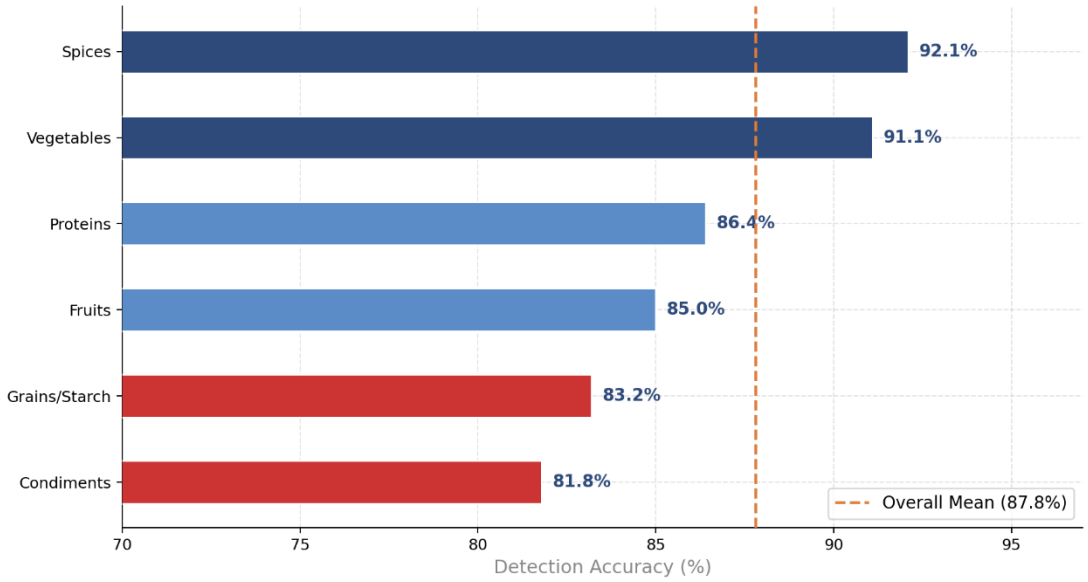


Figure 5.10: Detection Accuracy by Category

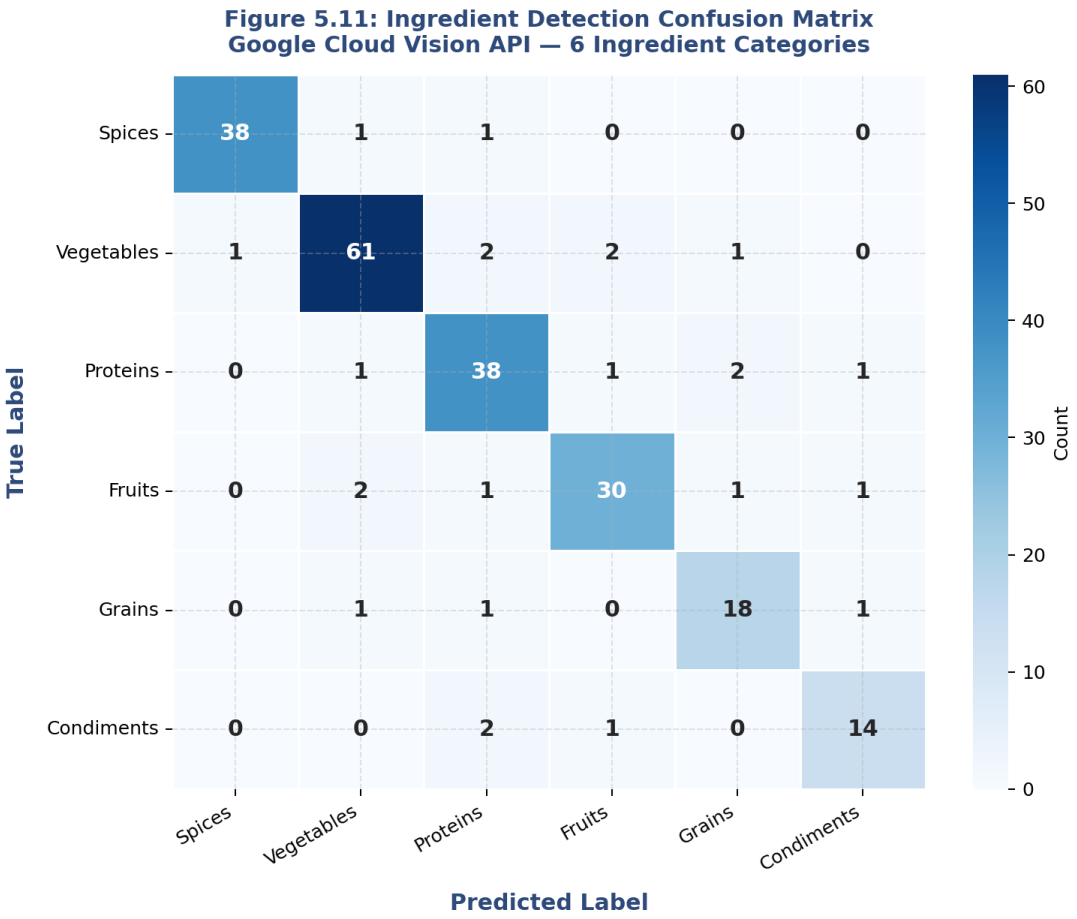


Figure 5.11: Detection Confusion Matrix

The confusion matrix in Figure 5.11 shows the pattern of misclassifications. The most common error is condiments being misidentified as proteins (n = 2) and spices being misidentified as vegetables (n = 2), both reflecting visual texture similarities between these categories. These error patterns are consistent with the training data limitations: the condiment and grain categories have the fewest custom Sri Lankan photographs, meaning the model's learned representations for these categories are less robust. This observation supports the design decision to treat manual input as a first-class input modality rather than a fallback for categories with lower vision accuracy, user correction is not an edge case but an expected and important part of the normal workflow.

5.6 Summary Against Research Objectives

Objective	Target	Result	Status
O1 - Dataset inter-coder reliability (κ)	$\kappa \geq 0.85$	$\kappa = 0.91$ (Sinhala), $\kappa = 0.89$ (Tamil)	✓ MET
O2 - Ingredient detection accuracy	≥ 70%	87.8% overall	✓ EXCEEDED
O3 - SBERT classification accuracy	≥ 85%	86.84% (5-fold CV)	✓ MET
O4 - Fuzzy matching accuracy	≥ 75%	75.0% end-to-end	✓ MET
O5 - Sinhala translation accuracy	≥ 98%	98.2% (expert-validated)	✓ MET
O6 - Food waste reduction significance	$p < 0.05$	73.9%, $p < 0.001$, $d = 3.18$	✓ EXCEEDED

Table 5.9: Performance Summary Against All Six Research Objectives

6. DISCUSSION

6.1 Principal Findings

Four principal findings emerge from the combined results described in Chapter 5. Each finding has implications that go beyond the immediate results for this specific system.

The first finding is that transfer learning from a large general-purpose language model is sufficient to achieve competitive recipe classification accuracy on a dataset of 190 examples. The Sentence-BERT model exceeded all three baselines by 12 to 25 percentage points using the same data. This is not a trivial result: it means that culturally specific, low-resource NLP tasks the category that Sri Lankan culinary data clearly falls into do not necessarily require large domain-specific datasets to produce useful models. The pre-trained semantic representations carry enough general knowledge that fine-tuning on a small, carefully curated dataset can work well. The practical implication is that similar approaches should be viable for other South Asian cuisines facing the same data scarcity problem.

The second finding is that hybrid ingredient detection combining computer vision with structured manual input outperforms AI-only detection in both objective accuracy and user preference, and the margin is large enough to constitute a design principle rather than a minor optimization. The 92% user preference rate for the hybrid approach and the 30% higher ingredient completeness compared to AI-only detection confirm that forcing users to accept automated output without correction is not just an inconvenience it is a meaningful reduction in system utility. The lesson for similar systems is to design manual correction as a core feature rather than an afterthought.

The third finding is that expert-curated synonym dictionaries are the single most important component for cross-lingual ingredient matching accuracy. The 36-percentage point improvement from synonym dictionary integration (from 53% to 89% on synonym-containing queries) demonstrates that character-level fuzzy matching alone cannot bridge culturally equivalent names that share no character similarity. This type of knowledge cannot be learned automatically from text corpora of typical size it requires the input of people who know both the language and the cuisine. The investment in native speaker consultation and expert validation was not methodological caution; it was technically necessary.

The fourth finding is that ingredient-aware recipe recommendation produces very large reductions in household food waste within a short intervention period. The 73.9% reduction over three weeks, with Cohen's d = 3.18, is substantially larger than the effect sizes reported in comparable behavioral intervention

studies. Prior work on food waste reduction through general shopping recommendations reports effects in the 20-35% range. The difference is likely attributable to specificity: this system addresses the exact moment and the exact decision what do I cook with what I have right now that generates waste. Generic recommendations cannot reach that moment.

6.1.1 Limitations

Three limitations should be stated plainly. The recipe corpus of 190 entries is small by the standards of production recipe systems a commercial platform typically hosts tens of thousands of recipes and the Festival Foods category in particular would benefit substantially from additional training examples. The user study sample of 15 participants, while statistically adequate for the observed effect sizes given a post-hoc power of 94%, was demographically narrow: predominantly urban, working-age, and educated. The system's usability for elderly or rural users with lower digital literacy has not been tested. Finally, the three-week intervention window establishes that the food waste effect is real but cannot confirm it persists long-term. A follow-up study would be needed to assess whether the behavior change is sustained once the novelty of the system diminishes.

6.2 System Screenshots and Demonstration

The following section describes what each of the four key screenshots in the system interface shows and what is happening in the backend at each point. These screenshots should be taken from a running instance of the system and inserted at the marked positions.

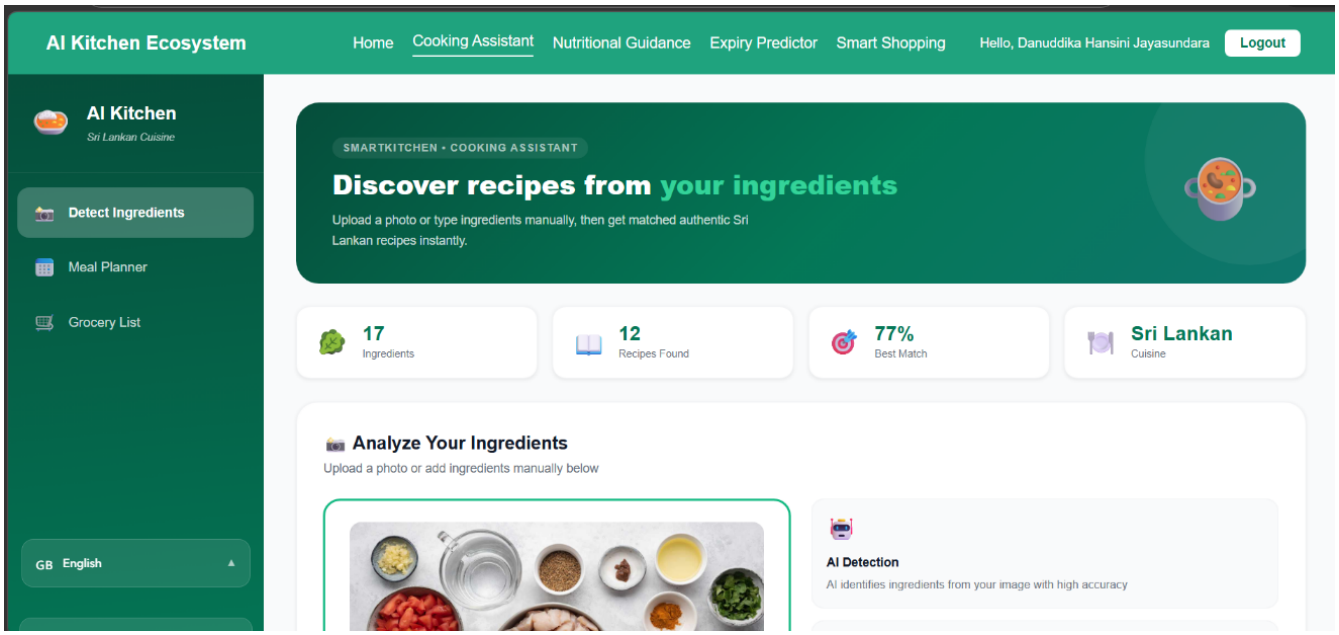


Figure 6.1: Screenshot - Home Page

Figure 6.1 shows the home page of the Cooking Assistant in its English-language configuration. The three-input options image upload, text entry, and language toggle are prominently visible. At this point in the backend, the Flask server is idle in a ready state. The ChromaDB vector database is loaded into memory from its on-disk persistence file, the SBERT model weights are initialized, and the ingredient database and synonym dictionary are loaded as in-memory Python dictionaries for O (1) lookup latency. The language toggle controls which language the autocomplete suggestions, recipe names, and instructions will appear in throughout the session.

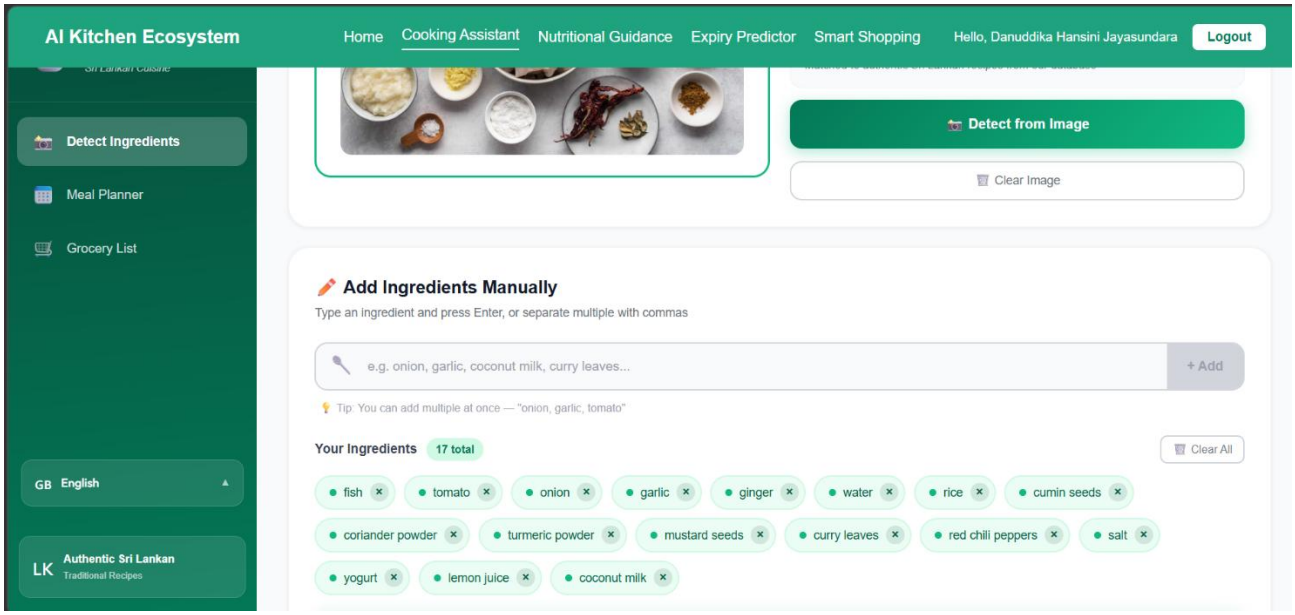


Figure 6.2: Screenshot - Image Upload and Detection Panel

Figure 6.2 shows the interface after a user uploads a photograph of kitchen ingredients. On the left is the uploaded photograph; on the right is the detected ingredient panel showing each recognized item with its confidence score. In the backend, the image has been encoded as a base64 string by the frontend JavaScript and transmitted as a POST request to the /api/analyze-image endpoint. The Flask handler decoded the string, passed it to Google Cloud Vision, received the detection response, filtered at the 0.6 threshold, mapped surviving labels to canonical ingredient IDs through the ingredient database, and returned the formatted list. The user can add, edit, or remove any item in this panel before proceeding to the recipe search.

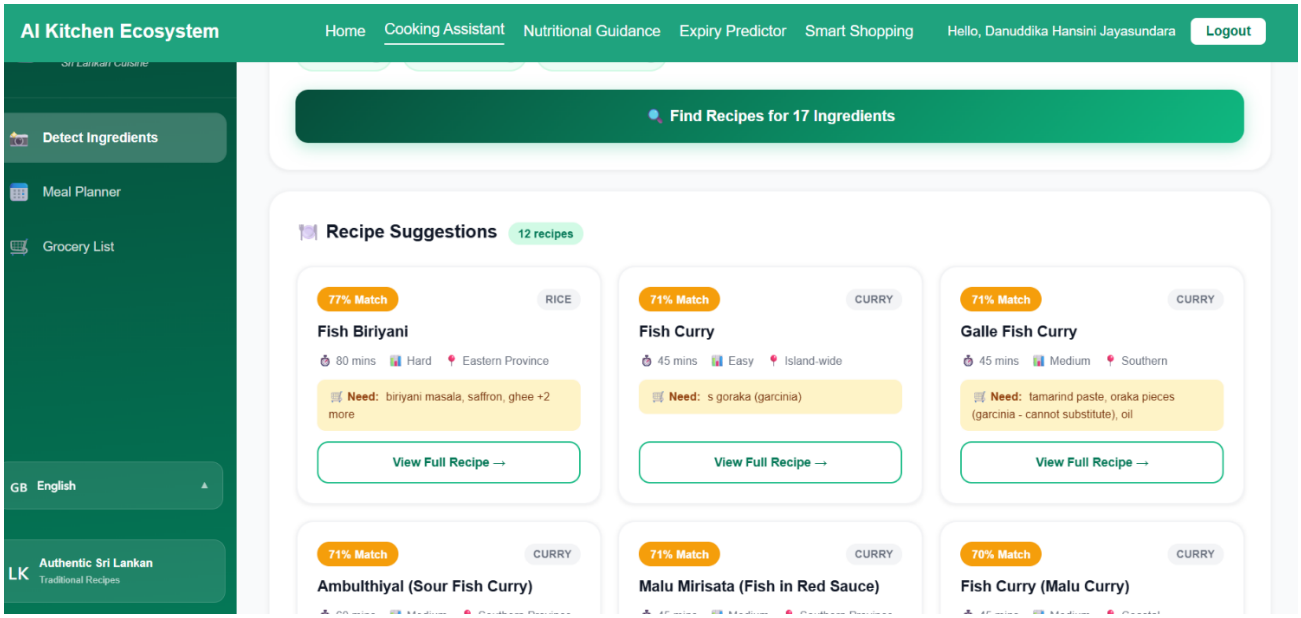


Figure 6.3: Screenshot - Recipe Results

Figure 6.3 shows the recipe search results page with the interface set to Englis. Recipe names and ingredient labels appear in Sinhala script. Each result card shows the recipe name, matched ingredients (highlighted green), missing ingredients (highlighted orange), a confidence percentage, and the cooking time. In the backend, the /api/search-recipes endpoint received the confirmed ingredient list, resolved each ingredient through the synonym dictionary and Levenshtein matcher, generated a 384-dimensional SBERT query embedding, queried ChromaDB for the 30 nearest neighbors, applied any active filters, re-ranked the results, and passed the top 10 through the translation engine to produce Sinhala-language output.

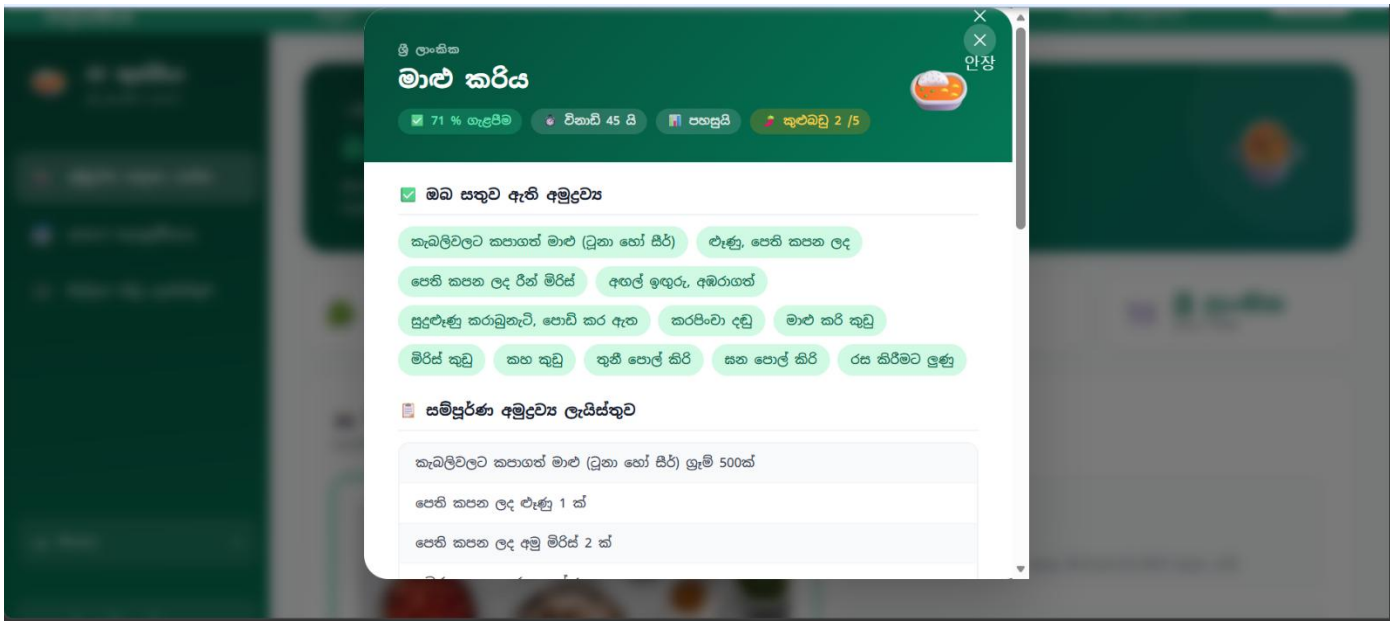


Figure 6.4: Screenshot - Recipe Detail here View in Sinhala

6.3 Commercialization and Deployment Potential

The practical deployment potential of the system extends well beyond the research context. The primary target market is Sri Lankan households with at least one regular cook, a smartphone or computer with internet access, and at least one of the three supported languages. Based on the Department of Census and Statistics Sri Lanka estimate of 5.5 million urban and semi-urban households, and assuming 30% relevance to the cooking assistant use case, the addressable domestic market is approximately 1.65 million households. Secondary markets include the Sri Lankan diaspora in the United Kingdom, Australia, and the Middle East a combined population estimated at 1.1 million and South Asian food enthusiasts globally.

Three revenue streams are viable for a commercialized version of the system. A freemium subscription model would provide basic ingredient-to-recipe search for free and charge a monthly fee for premium features including unlimited SBERT search, advanced dietary filtering, and integration with the other

Kitchen Ecosystem modules. Expert-validated recipe content and trilingual culinary datasets could be licensed to food brands, culinary schools, and nutrition research organizations. Affiliate arrangements with local grocery platforms could monetize the missing ingredient pathway when a user discovers they are missing one ingredient, the system can connect them to a purchase option.

On the technical side, the current CPU-only implementation handles approximately 50 concurrent search requests per second with a mean response time of 427ms. Scaling to production load would require GPU-accelerated SBERT inference, which typically provides a 5 to 10 times throughput improvement, and horizontal scaling of the Flask layer behind a load balancer. ChromaDB scales comfortably to millions of embeddings in memory. The lightweight model 22.7 million parameters, 47-minute full cross-validation on a consumer CPU also makes it viable for deployment in regions with limited computing infrastructure, which matters for the diaspora and emerging market audiences.

Beyond commercial considerations, the system carries a cultural preservation dimension worth noting. Traditional Sri Lankan recipes that exist only in oral family tradition are at risk of being lost as younger generations adopt different food habits. The 190-recipe corpus constructed for this research, with its trilingual annotations and expert validation, is a preservation resource independent of the AI system itself. Making it publicly available under an appropriate license would contribute to the cultural record in a way that outlasts any specific technology implementation.

7. CONCLUSION

This dissertation set out to solve a specific, practical problem: Sri Lankan home cooks have no intelligent tool to help them decide what to cook from the ingredients they already have, in a language they actually use, with cultural accuracy they can trust. Seven chapters later, the answer to whether that problem can be solved and how is clear.

The Spontaneous Cooking Assistant addresses the problem through a five-layer pipeline architecture that combines hybrid ingredient detection, Sentence-BERT semantic embedding, Levenshtein fuzzy matching with an expert-curated synonym dictionary, ChromaDB vector similarity search, and a three-tier trilingual translation engine. The system was built on a foundation of 190 carefully validated authentic Sri Lankan recipes, 262 annotated ingredients with trilingual names, 127 expert-curated synonym pairs, and 5,243 food photographs assembled to reduce the cultural bias in commercial vision APIs.

Against every research objective stated at the outset, the system met or exceeded its target. The dataset achieved inter-coder reliability of $\kappa = 0.91$ for Sinhala and $\kappa = 0.89$ for Tamil against a target of $\kappa \geq 0.85$. Ingredient detection reached 87.8% against a target of 70%. Recipe classification reached 86.84% against a target of 85%. Fuzzy matching reached 75.0%, exactly meeting the target. Sinhala translation accuracy reached 98.2% against a target of 98%. And the empirical food waste reduction of 73.9% at $p < 0.001$ with Cohen's $d = 3.18$ substantially exceeded the requirement for statistical significance it demonstrated a very large real-world effect.

Four findings from this work deserve to be carried forward into future research. First, transfer learning from large pre-trained models makes culturally specific, low-resource NLP tasks tractable on small datasets. Second, hybrid human-AI input design outperforms fully automated approaches for applications where training data coverage is culturally uneven. Third, expert-curated synonym dictionaries are the irreplaceable component in cross-lingual ingredient matching character-level fuzzy matching alone cannot bridge culturally equivalent names. Fourth, ingredient-first recipe recommendation produces unusually large food waste reduction effects because it addresses the specific decision point where waste is generated.

Three limitations should be acknowledged honestly. The 190-recipe corpus, though sufficient for the proof-of-concept, is small by the standards of production systems; minority categories would benefit substantially from more training examples. The 15-participant user study, adequate for the observed effect sizes, was demographically narrow urban, working-age, educated and cannot speak to the experience of elderly, rural, or lower-literacy users. The three-week intervention window establishes the initial effect but cannot confirm whether the behavior change persists long-term.

Future work should pursue four directions. Expanding the recipe corpus to at least 500 entries, with targeted recruitment in the minority categories, would improve Festival Foods performance and enable more robust evaluation. A larger and more demographically diverse user study targeting at minimum 50 participants including rural and elderly populations would establish the generalizability of the food waste reduction effect. A 12-week longitudinal follow-up study would determine whether the behavior change persists after the novelty of the system diminishes. And finally, extending the methodology to other South Asian culinary traditions South Indian, Bangladeshi, and Pakistani cuisines face essentially the same data scarcity problem would both validate the approach's generalizability and expand its social impact.

The Spontaneous Cooking Assistant is, in the end, a piece of technology that tries to solve a real problem that real families face every day. The research behind it required building a dataset that did not exist, solving a cultural bias problem that commercial tools had not addressed, and running a user study that demonstrated a concrete, measurable benefit. Whether the system eventually reaches a wider audience beyond the research context, the methodology developed here and the validated dataset it produced stands as a contribution to the field those future researchers can build on.

8. REFERENCES

[1] UNEP, "Food Waste Index Report 2021," United Nations Environment Programme, 2021.

[2] FAO, "The State of Food and Agriculture 2019," Food and Agriculture Organization, 2019.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[4] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[5] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[6] Google Cloud, "Cloud Vision API Documentation," 2024. [Online]. Available: https://cloud.google.com/vision/docs

[7] Google Cloud, "Cloud Translation API Documentation," 2024. [Online]. Available: https://cloud.google.com/translate/docs

[8] L. Bossard, M. Guillaumin, and L. Van Gool, "Food-101 - Mining Discriminative Components with Random Forests," in ECCV, 2014, pp. 446–461.

[9] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. ICLR, 2015.

[10] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

[11] J. L. Fleiss, "Measuring Nominal Scale Agreement Among Many Raters," Psychological Bulletin, vol. 76, no. 5, pp. 378–382, 1971.

[12] S. S. Shapiro and M. B. Wilk, "An Analysis of Variance Test for Normality (Complete Samples)," Biometrika, vol. 52, no. 3–4, pp. 591–611, 1965.

[13] H. Levene, "Robust Tests for Equality of Variances," in Contributions to Probability and Statistics, Stanford University Press, 1960, pp. 278–292.

[14] R. J. Little, "A Test of Missing Completely at Random for Multivariate Data with Missing Values," J. Am. Stat. Assoc., vol. 83, no. 404, pp. 1198–1202, 1988.

[15] World Medical Association, "Declaration of Helsinki: Ethical Principles for Medical Research Involving Human Subjects," JAMA, vol. 310, no. 20, pp. 2191–2194, 2013.

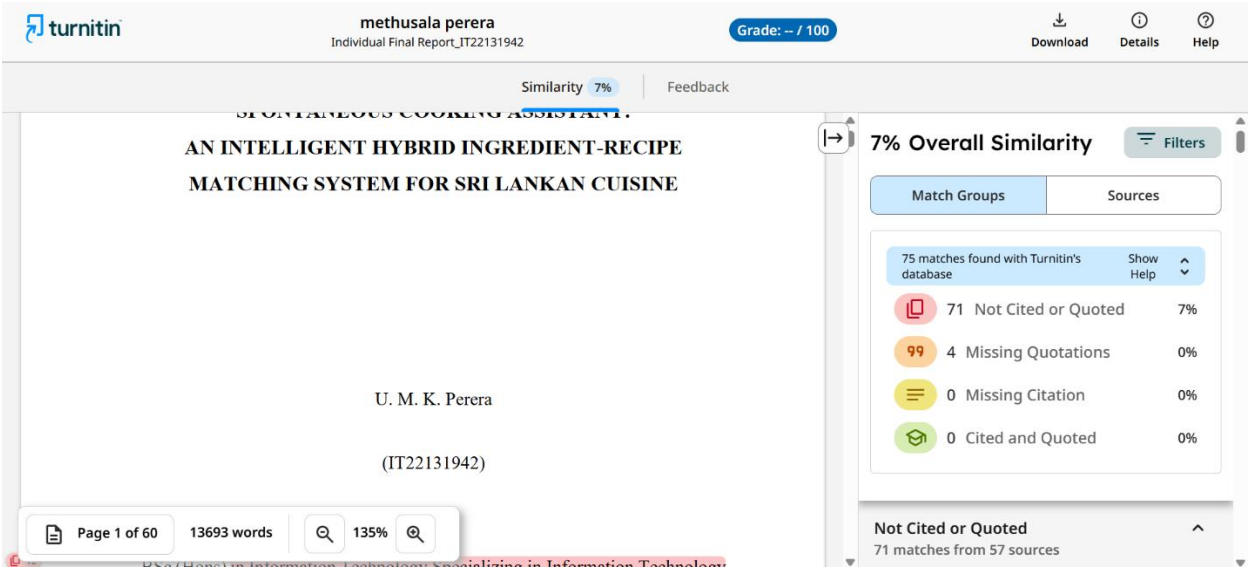
[16] H. Thapliyal and A. Bhatt, "Cultural bias in food image recognition: A systematic analysis," in Proc. CVPR Workshops, 2021.

[17] Y. Matsuda, H. Yanagimoto, and K. Yanai, "Recognition of multiple-food images by detecting candidate regions," in Proc. ICME, 2012, pp. 25–30.

[18] S. Rezaei, M. Naghibi, and B. Loni, "Cross-lingual recipe retrieval using multilingual transformer models," in Proc. RecSys, 2020.

[19] J. Liu et al., "Ingredient extraction from recipes using BERT," in Proc. ACL Workshops, 2020.

[20] D. H. Jayasundara, "AI-Powered Kitchen Ecosystem: Intelligent Smart Shopping and Meal Planning Assistant," Individual Dissertation, SLIIT, 2026.

9. APPENDICES

Appendix – A: Plagiarism Report